

所属类别	2026 年“华数杯”全国大学生数学建模竞赛	参赛编号
本科组		CM2602322

面向 VLSI 布图规划的四层统一求解框架

摘 要

超大规模集成电路 (VLSI) 中的**布图规划**直接决定芯片的性能、面积与良率。本文面向 VLSI 布图规划中的多个子问题, 以模块位置、方向与形状为决策变量, 以**模块互不重叠、不超出芯片轮廓**为核心约束, 围绕轮廓面积与总半周长线长 (HPWL) 两类目标建立优化模型, 并构建了“多源初解生成—并行元启发优化—双向反馈定界校验—场景适配”的四层求解框架, 综合运用**模拟退火、迭代局部搜索、B*-树、约束规划 (CP-SAT)**等方法, 使各问题结果同时具备可行性证书、下界估计与可量化的最优性差距。

轮廓自由且无连接的问题一, 被建模为**二维矩形装箱的字典序双目标优化**: 以面积最小为一级目标、长宽比接近 1 为二级目标, 并论证了问题的 NP-hard 性质与整数坐标最优解的存在性; 以 Skyline-迭代局部搜索为主、B*-树-模拟退火为对照。**结果表明: 三组轮廓面积分别为 192818、191026、291311, packing 密度达 91.97%–93.77%, 长宽比介于 1.065–1.093 之间。**

问题二在固定正方形轮廓下优化连线: 建立**总 HPWL 最小化**的固定轮廓布图规划模型, 借助 HPWL 目标的凸分片线性结构构造无重叠松弛的全局下界, 以 Skyline-ILS/SA 为主算法、序列对-自适应遗传算法为对照。**结果表明: 三组总 HPWL 分别为 276865、536430、800202, 均满足无重叠与轮廓可行约束, 主算法较对照改进 22.8%–26.0%。**

问题三聚焦可行性的临界边界: 利用可行性关于 L 的单调性, 以**整数二分结合双证书夹逼** (启发式装箱给可行证书、CP-SAT 给不可行证书) 求最小死区比例, 并在最小轮廓下复用问题二模型更新 HPWL。**结果表明: 三组最小死区比例分别为 11.8%、8.2%、7.1%, 较问题二的 15% 显著压缩, 总 HPWL 仅上升 2.2%–5.9%。**

问题四将模块推广至 L/T 型异形: 通过**矩形切割组合刚性链接**使修正模型与问题一逐条对应、算法在问题一框架上适当修改即可复用。**结果表明: 4 个模块以 6×4 完美铺砌, $S^* = 24$ 、密度 100% 达到理论下界, 较包围盒近似节省 20% 面积。**

综上, 所建四层求解框架在轮廓面积、总 HPWL、最小死区比例与异形模块等目标下均取得高质量、可验证的结果, 模型贴合实际、算法高效、可扩展性强, 可为大规模芯片布图规划提供有效的求解方案。在 $n = 300$ 的最大规模下, 各问题单次求解以 6 核并行、按墙钟时间 (wall-clock time) 计时平均耗时约 2 分钟, 具备良好的工程实用性。

关键词: VLSI 布图规划 矩形装箱 半周长线长 (HPWL) 模拟退火 B*-树 约束规划

一、问题的提出

1.1 问题的背景

布图规划 (Floorplanning) 是超大规模集成电路 (VLSI) 物理设计流程中的关键环节, 其任务是在给定芯片轮廓内确定各功能模块的位置与方向, 使模块互不重叠、不超出轮廓, 并最小化芯片面积与模块间连线长度等目标。布图质量直接影响芯片的面积、性能、良率与可靠性, 是电子设计自动化 (EDA) 领域最核心的研究问题之一。布图规划属于 NP-hard 问题, 精确算法仅适用于小规模实例, 工程上普遍采用构造式启发式与模拟退火 (SA) 等元启发式方法求解^[4]。随着大规模 ASIC/SoC 采用分层设计与固定轮廓约束, 布图规划从“最小化面积”转变为“在给定轮廓内最小化总连线长度”的约束优化问题^[5]; 半周长线长 (HPWL) 因计算简便且与真实布线长度高度正相关, 成为最常用的线长评估指标。近年来, 基于泊松方程的解析方法 (PeF)^[6] 与深度强化学习布图 (AlphaChip)^[7]、面向多设计规则的三维布图学习 (RulePlanner)^[8] 进一步拓展了大规模布图的求解范式, 为布图规划的高效求解提供了新的思路。

1.2 问题重述

(1) 问题一: 问题一要求在芯片轮廓不固定且模块之间无任何连接关系的前提下, 确定各矩形功能模块在芯片内的摆放位置与方向, 并且允许各个模块进行 90° 旋转, 使包围所有模块的**芯片轮廓面积最小**; 当轮廓面积相同时, 要求其**长宽比尽量接近 1**。需根据所建立的模型与 `.blocks` 文件信息, 对附件中的 `n100`、`n200`、`n300` 三组芯片分别独立完成模块摆放, 并给出三组芯片轮廓的面积、长宽比及模块摆放的可视化结果。

(2) 问题二: 问题二要求在**芯片轮廓固定为正方形且模块之间存在连接关系**的前提下, 确定各矩形功能模块在芯片内的摆放位置与方向, 并且允许各个模块进行 90° 旋转, 使其**模块之间总半周长线长 (HPWL) 最小**, 同时满足**模块之间互不重叠、不超出芯片轮廓**的约束条件。需根据所建立的模型与附件信息, 假设死区比例为 **0.15**, 对附件中的 `n100`、`n200`、`n300` 三组芯片分别独立完成模块摆放, 并给出三组芯片的总连线长度及模块摆放的可视化结果。

(3) 问题三: 在**问题二**的基础上, 为满足**模块互不重叠、不超出芯片轮廓**的约束条件, 求解**死区比例的最小值**, 即求解使得存在一个可行布局方案的最小死区比例 (随着死区比例减小, 芯片轮廓尺寸逐渐减小, 布图越难以满足约束条件)。需分别给出附件中 `n100`、`n200`、`n300` 三组芯片的死区比例最小值, 并基于**问题二所建立的模型**与死区比例最小值, 更新问题二的总 HPWL 及模块摆放的可视化结果。

(4) 问题四: 在**问题一**的基础上, 假设模块不再全是规则的矩形, 而可能是 **L 型和 T 型异形模块**, 且所有模块均可进行 90° 180° 270° 旋转。需讨论**问题一**所建立的模型应如何修正 (求解算法可在问题一算法基础上适当修改, 不要求重新设计新的优

化算法)。为验证修正后模型的有效性，现给出 4 个待摆放模块，需基于修正后的模型求解如何摆放这 4 个模块使包围它们的芯片轮廓面积最小，并给出最优摆放示意图及此时的最小面积。

二、问题分析

本文四个子问题统一在”构造可行解 → 提升解质量 → 严格定界 → 最优性闭环”的数学逻辑主线之下：以启发式搜索给出可行解（上界 UB），以下界估计（LB）刻画与最优解的差距，并通过 gap 报告量化最优性，其总体逻辑如图图 1所示；各问题在此基础上再按”多源初解—并行元启发优化—双向反馈定界校验—场景适配”四层框架落地。

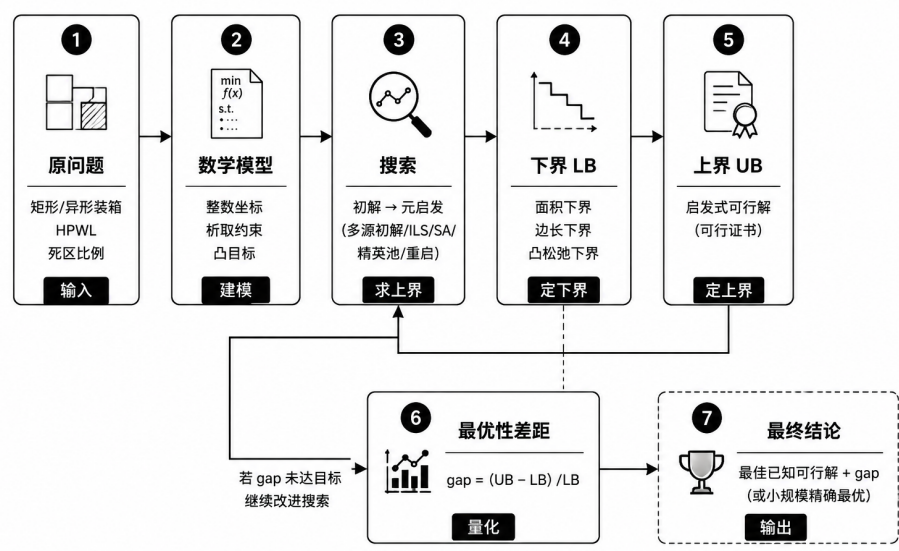


图 1 总体数学逻辑：从启发式求解到可验证定界

2.1 问题一分析

问题一要求在轮廓自由、模块之间无任何连接的前提下，确定各矩形模块（允许 90° 旋转）的位置与方向，使包围所有模块的轮廓面积最小，面积相同时长宽比尽量接近 1。从数学形式上看，该问题可归入经典的**二维矩形装箱 (Rectangle Packing) 问题**——将给定矩形集互不重叠地摆放并确定其最小包围盒；由于正方形装箱是它的特例，问题属于 **NP-hard**，在 n 较大时难以在多项式时间内精确求解，只能借助构造式启发与元启发式策略逼近。

同时，该问题存在两个优化目标：主目标为**最小化芯片轮廓面积**，次目标为**在面积相同时使长宽比尽量接近 1**。因此，采用**字典序优化策略**：先求解面积最小的布局，

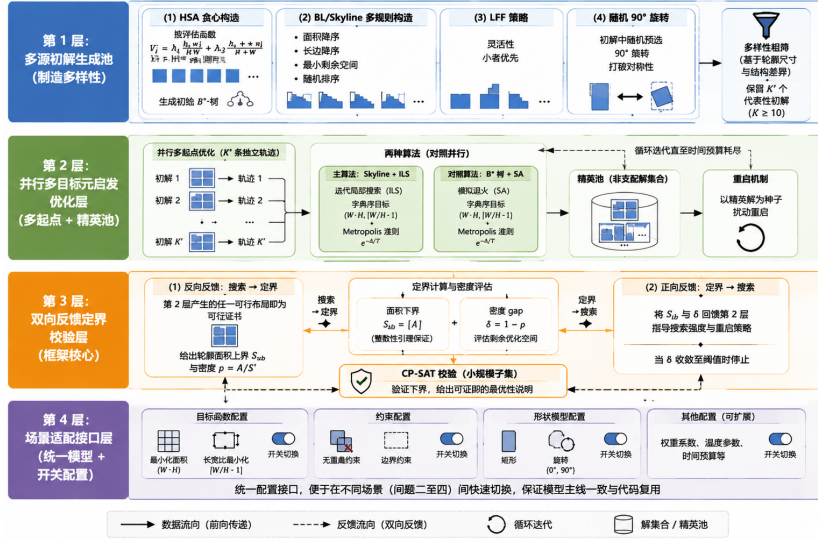


图 2 问题一四层实施框架

再在当前获得的最小面积解集中进一步搜索长宽比最接近 1 的布局 (作为次目标的启发式近似), 严格符合题意”面积优先、长宽比次之”的要求。

基于上述分析, 针对问题一制定**四层实施框架**, 其结构如图 2所示, 各层环环相扣、以具体算法与数据流落地, 并以实验结果闭环验证。

2.2 问题二分析

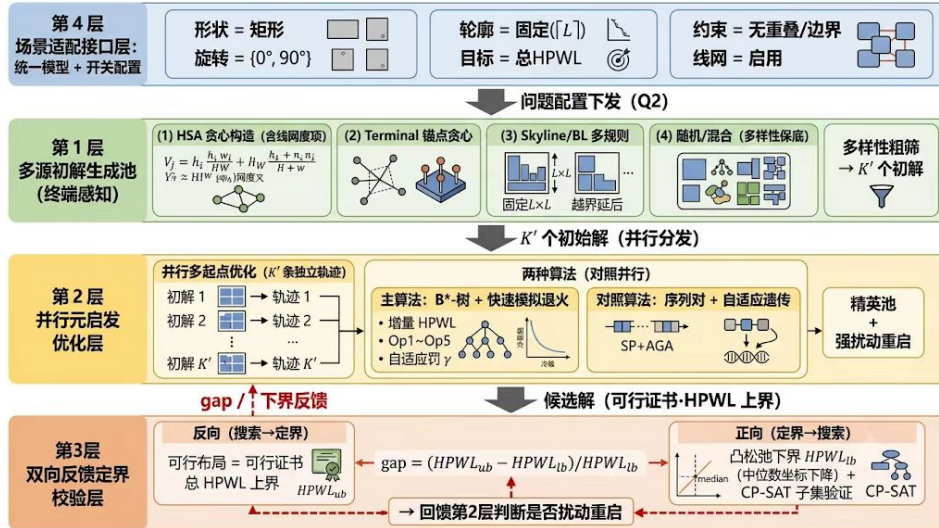


图 3 问题二四层实施框架

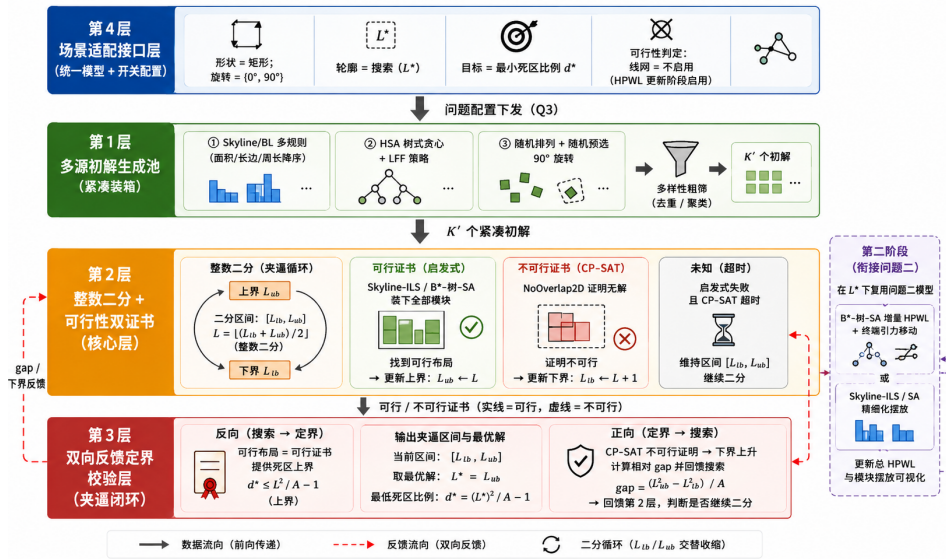
问题二则引入了固定正方形轮廓与模块间连接关系: 在死区比例 $d = 0.15$ 给定的正方形芯片内, 确定各矩形模块 (允许 90° 旋转) 的位置与方向, 使所有线网的**总半**周长线长 (HPWL) 最小, 同时满足模块互不重叠、不超出轮廓的约束。就其数学本质而言, 该问题属于**固定轮廓布图规划 (Fixed-Outline Floorplanning)**^[5]——在给

定轮廓内无重叠摆放矩形模块并最小化连线长度；由于二维装箱可归约为其特例，问题同样是 **NP-hard**，大规模情形下必须依赖启发式方法。

同时，与问题一相比，本问的芯片轮廓不再是可以自由优化的变量，而是**给定的硬约束**：空白区从”优化目标”退化为”固定约束”，目标为单一的**最小化总 HPWL**，需与**模块互不重叠、不超出轮廓**的约束同时满足。此外，附件中 Terminal 的坐标全部位于轮廓边界，构成围绕芯片的**边界引力源**，将与之相连的模块拉向芯片四周，与不重叠约束形成对抗，这是本问优化的核心难点。

基于上述分析，针对问题二在问题一框架基础上沿用**四层实施框架**，其结构如图图 3所示：由第 4 层场景适配接口将配置切换为”轮廓固定、目标 HPWL、线网启用”，第 1-3 层分别以”终端感知多源初解、并行元启发优化（Skyline-ILS/SA 主、B*-树-快速 SA 与 SP-AGA 对照）、凸松弛下界与可行性证书的双向定界”落地，各层环环相扣、以具体算法与数据流闭环验证。

2.3 问题三分析



问题三在问题二的基础上，转而求解**最小死区比例 d^*** ，即最小可行正方形边长 $L^* = \sqrt{A(1 + d^*)}$ 。从数学角度看，问题转化为**正方形装箱的可行性判定 (Square Packing Decision Problem)**：判断给定的矩形模块集能否在 $L \times L$ 正方形内无重叠摆放，并在最小可行边长处逼近死区比例的临界值。其判定版本为 **NP-complete**，大规模时须以启发式构造可行解、以约束规划证明不可行性，两者互补。

同时，与问题一、问题二不同，本问的优化对象既不是轮廓面积也不是总连线长度，而是**可行性的临界轮廓**：随着 L 减小，空白区被压缩，模块越难以同时满足不重叠与不超出轮廓的约束。其关键性质在于可行性关于 L 的**单调性**——若 L 可行则

$L + 1$ 必然可行，故可在整数 L 上二分搜索最小可行边长。需要强调的是，本文仅将 CP-SAT 在规定时间内返回不可行 (INFEASIBLE) 作为严格不可行证书：其中对面积最大的前 15 个模块判定子集不可行，即可证明整体不可行；启发式仅在找到完整可行布局时给出可行证书（收缩上界），启发式失败不视为不可行；CP-SAT 超时 (UNKNOWN) 则不更新区间。据此以”可行证书 + 不可行证书”双向夹逼，将最小死区比例的求解转化为可验证的搜索。

基于上述分析，针对问题三沿用**四层实施框架**，其结构如图图 4所示：由第 4 层场景适配接口将配置切换为”轮廓搜索 L^* 、目标最小死区比例、可行性判定不启用线网”，第 1-3 层分别以”紧凑装箱多源初解、整数二分与可行性双证书、双向反馈夹逼”落地；求得 L^* （即 d^* ）后，再在 L^* 下复用问题二所建立的模型与算法更新总 HPWL 及模块摆放可视化，各层环环相扣、以具体算法与数据流闭环验证。

2.4 问题四分析

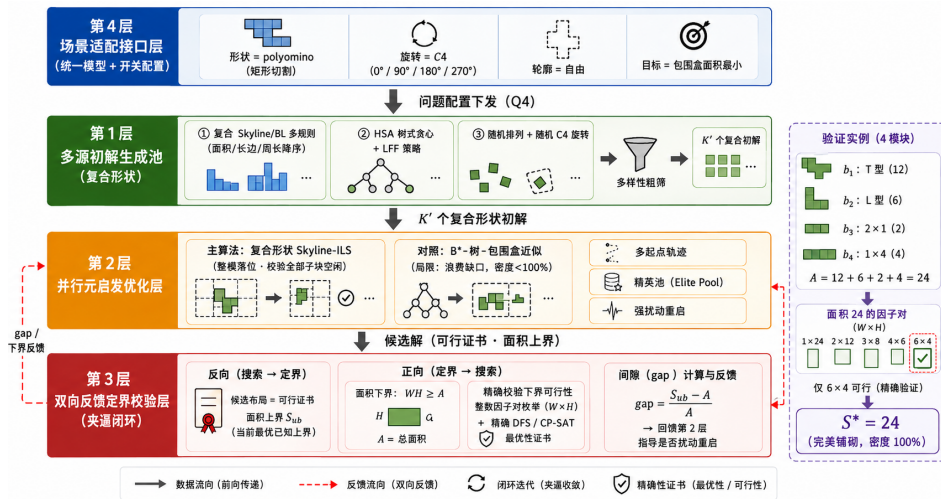


图 5 问题四四层实施框架

问题四在问题一的基础上,将模块推广为 L 型、T 型等异形模块并允许 90° 180° 270° 旋转, 核心任务在于讨论问题一模型应如何修正, 并验证修正模型的有效性。从数学结构上看, 该问题对应**异形模块装箱 (Polyomino Packing)**——将给定异形模块集无重叠摆放并确定其最小包围盒; 它同样包含矩形装箱为特例, 属 **NP-hard**, 但验证实例仅含 4 个模块, 规模极小, 借助精确搜索即可获得可证明的最优解。

同时,与问题一相比,本问的核心是**模型修正**:矩形模块可用二元旋转直接描述,而异形模块需先将每个模块**切割**为若干矩形子块, 同一模块的子块间通过**刚性链接 (硬约束)**保持固定相对位移, 随模块整体 C_4 旋转同步变换; 不同模块的子块之间沿用问题一的非重叠约束。这一”矩形切割 + 刚性链接”的表述使修正模型与问题一逐条对应, 可在问题一算法基础上适当修改后复用。

基于上述分析，针对问题四沿用**四层实施框架**，其结构如图图 5所示：由第 4 层场景适配接口将配置切换为”形状 =polyomino（矩形切割）、旋转 =C4、轮廓 = 自由、目标 = 面积最小”，第 1-3 层分别以”复合形状多源初解、复合形状 Skyline-ILS (B*-树-包围盒近似对照)、面积下界与精确 DFS 定界”落地，各层环环相扣、以具体算法与数据流闭环验证。

三、模型的假设

为使模型在保持物理意义的同时便于求解，作如下假设：

假设 1（模块刚性）：模块为刚体，仅可平移与旋转；问题四的异形模块可视为若干矩形子块的刚性组合。

假设 2（不重叠与轮廓）：模块间不允许重叠（允许共边/共点）；芯片轮廓为轴平行矩形，其中问题一、四轮廓自由，问题二、三为正方形轮廓。

假设 3（Terminal 规则）：Terminal 为固定焊盘，仅作为固定引脚参与 HPWL 计算，不占面积、不参与重叠与边界约束。

假设 4（引脚与线长）：模块引脚位于几何中心；线网线长以半周长线长（HPWL，即引脚包围矩形半周长）估算，与引脚内部连线方式无关。

假设 5（整数性）：模块尺寸与坐标为整数——问题一、三由整数性引理保证存在整数最优解，问题四的格点/子块模型天然取整数坐标。

四、符号说明

表 1 变量符号及其解释说明

符号	解释说明	符号	解释说明
$\mathcal{B}, \mathcal{T}, \mathcal{N}$	模块集、Terminal 集、线网集	$\mathcal{P}_k$	线网 k 的引脚集合
$w_i, h_i; \hat{w}_i, \hat{h}_i$	模块 i 原始/旋转后宽高	r_i	模块 i 旋转标志 (问题四取 C_4)
$(x_i, y_i); (x_m, y_m)$	模块 i 左下角坐标/模块 m 锚点	$W, H; S = WH$	轮廓宽高、轮廓面积
A	模块总面积	S^*, ρ	最优面积、packing 密度 A/S
$d; L = \sqrt{A(1+d)}$	死区比例、正方形轮廓边长	L^*, d^*	最小可行边长/死区比例 (问题三)
L_{lb}	边长理论下界	X_t, Y_t	端子 t 固定坐标
$HPWL_k$	线网 k 半周长线长	$HPWL_{lb}$	凸松弛下界
$K_m, (w_{m,k}, h_{m,k})$	模块 m 子块数/子块尺寸 (问题四)	$(a_{m,k}, b_{m,k}), (x_{m,k}, y_{m,k})$	子块局部偏移/绝对坐标
$\lambda_1, \lambda_2, \lambda_3$	HSA 评估权重系数	$K, K'; T$	初解数、粗筛保留数; 退火温度

五、模型的建立与求解

5.1 问题一

5.1.1 模型的建立

由于芯片是由各个模块拼接而来, 因此设模块集合 $\mathcal{B} = \{1, 2, \dots, n\}$, 模块 i 的原始宽高为 (w_i, h_i) ; 决策变量为模块 i 左下角坐标 (x_i, y_i) 与旋转标志 $r_i \in \{0, 1\}$, 其中 $r_i = 1$ 表示旋转 90° , 旋转即交换宽高。旋转后模块实际宽高为

$$\hat{w}_i = (1 - r_i)w_i + r_i h_i, \quad \hat{h}_i = r_i w_i + (1 - r_i)h_i \quad (1)$$

设芯片轮廓宽、高分别为 W, H , 模块总面积为 $A = \sum_{i \in \mathcal{B}} w_i h_i$, 芯片轮廓总面积为 $S = W \cdot H$ 。

又因为在芯片制作是要求模块互不重叠, 即 $\forall i, j \in \mathcal{B}, i < j$, 则下述四个条件至少满足其一:

$$(x_i + \hat{w}_i \leq x_j) \vee (x_j + \hat{w}_j \leq x_i) \vee (y_i + \hat{h}_i \leq y_j) \vee (y_j + \hat{h}_j \leq y_i) \quad (2)$$

轮廓边界约束：

$$0 \leq x_i, \quad x_i + \hat{w}_i \leq W, \quad 0 \leq y_i, \quad y_i + \hat{h}_i \leq H, \quad \forall i \in \mathcal{B} \quad (3)$$

上述非重叠约束与轮廓边界约束的几何含义如图图 6所示：相邻模块在水平或竖直方向上必须保持分离以保证互不重叠，同时所有模块整体必须位于宽为 W 、高为 H 的芯片轮廓之内。

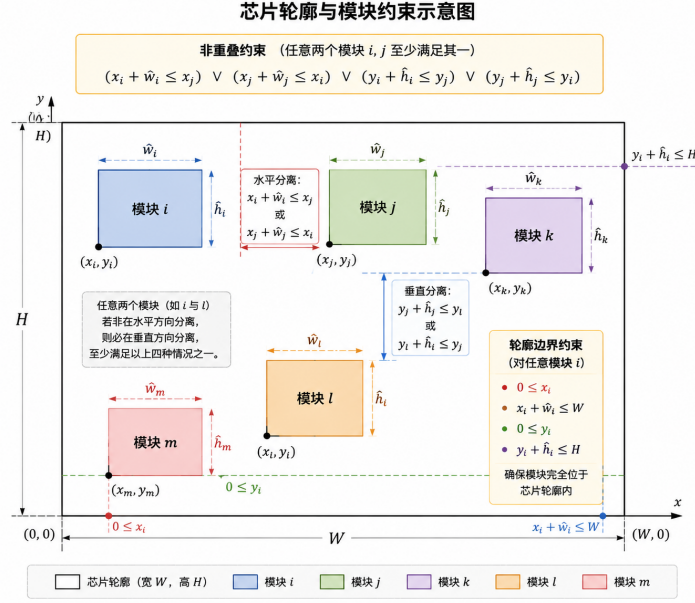


图 6 问题一约束条件示意图

在布图规划中，评价布局优劣的首要指标是芯片轮廓面积。由于题目要求在面积相同的情况下长宽比越接近 1 越好，两个目标之间存在明确的主次关系，不宜直接加权求和，故本文采用**字典序优化策略**：先最小化轮廓面积 $S = W \cdot H$ ，即

$$\min f_1 = W \cdot H \quad (4)$$

在得到最优面积 S^* 后，再在面积最优解集中进一步优化长宽比，即

$$\min f_2 = \left| \frac{W}{H} - 1 \right| \quad \text{s.t. } WH = S^* \quad (5)$$

其中 S^* 为式((4) 式)的最优值，上述两式构成字典序双目标，严格符合题意”**面积优先、长宽比次之**”的要求。

5.1.2 理论分析

为给求解算法的设计提供严谨的理论依据，本小节从计算复杂性、解空间结构与性能界三个层面，对问题一的模型性质进行分析，依次给出 NP-hard 性质、整数性引理与面积下界三个结论，并据此确定”启发式为主、精确法为辅”的求解策略。

(1) **NP-hard 性质**。该问题包含经典二维矩形装箱 (Rectangle Packing) 问题为特例。由于将正方形装箱判定问题可多项式归约到本问题，故本问题属于 NP-hard 问题。这意味着当模块规模 n 较大时，问题的解空间随 n 呈阶乘级爆炸，精确算法 (如 MILP、约束规划) 在最坏情况下需要指数级时间，仅适用于小规模实例。因此在 $n = 100 \sim 300$ 的赛题规模下，无法依赖精确算法在合理时间内得到最优解，必须采用启发式与元启发式方法进行求解^[1]。这一结论直接决定了后续求解框架的总体方向，即第 1 层的多源初解构造与第 2 层的元启发式改进。

(2) **整数坐标存在性命题**。**命题**：若所有模块宽高均为整数，且允许模块平移与 90° 旋转，则存在一个最优布局，其所有模块左下角坐标均为整数。**证明思路**：从任一最优布局出发，在不改变模块间左右/上下分离关系的前提下进行连续压缩，使至少一个分离约束达到紧约束；重复该压缩过程直至不存在可压缩模块。此时每个模块的左边界 (下边界) 必然贴紧另一模块的右边界 (上边界) 或轮廓边界，故所有模块坐标均可表示为一系列整数宽度 (高度) 的有限和，因此均为整数^[2]。由此可得：对于本文的整数尺寸数据集，可将轮廓宽度、高度及问题三的正方形边长搜索限制在整数域，从而为第 3 层“在整数 S 的分解上枚举并检验可 packing 性”的定界校验提供理论支撑。

(3) **下界与密度**。由于各模块互不重叠且全部位于轮廓内部，模块总面积 $A = \sum_{i \in \mathcal{B}} w_i h_i$ 必然不超过轮廓面积，故面积最优值满足下界 $S^* \geq \lceil A \rceil$ 。据此定义 packing 密度 $\rho = A/S$ ($0 < \rho \leq 1$)，并以 $\text{gap} = 1 - \rho$ 衡量当前解与面积下界的差距：密度越接近 1，说明空白面积越少，布局质量越高。该指标既可用于求解过程中实时判断剩余优化空间，也可用于最终结果的量化评价，使启发式求解结果的优劣具有可度量的客观标准。

综上，NP-hard 性质决定了求解必须采用启发式方法，整数性引理将搜索空间离散化并提供了定界枚举的依据，面积下界与密度指标则为求解过程的迭代控制与最终结果的质量评价提供了量化准则。三者共同构成问题一求解框架的“搜索方向—搜索空间—搜索质量”理论闭环，为后续四层求解框架的设计奠定了完整的基础。

5.1.3 模型的求解

针对问题 NP-hard 与 $n = 100 \sim 300$ 的大规模特点，本文采用图 2 所示的四层求解框架，各层以具体算法与数据流落地。

第 1 层：多源初解生成池，制造多样性。

为了丰富初始解的多样性，该论文通过综合四种机制生成 $K \geq 10$ 个初始布局：

(1) **HSA 贪心构造**：按评估函数 $V_i = \lambda_1 h_i w_i / (HW) + \lambda_2 (h_i + w_i) / (H + W)$ 降序装箱生成紧凑的初始 B*-树^[1]；

(2) **BL/Skyline 多规则构造**：分别按面积降序、长边降序、最小剩余空间与随机排序四种优先级各生成布局；

(3) LFF 策略；

(4) 初解中随机预选 90° 旋转以打破对称性。

随后以轮廓尺寸与结构差异构造多样性指标，粗筛保留 K' 个代表性初解进入下一层。

第 2 层：并行多目标元启发优化层，多起点结合精英池。

在第一层中的 K' 个初始解传入第二层之后，对 K' 个初解各启动一条独立的元启发轨迹，并多核并行，以 Skyline 结合迭代局部搜索 (ILS) 为主算法、B*-树-模拟退火 (SA) 为对照算法^[3]；两者均采用字典序目标 $(W \cdot H, |W/H - 1|)$ ，并按 Metropolis 准则 $e^{-\Delta/T}$ 接受暂时劣解以跳出局部最优。各轨迹收敛后将解放入精英池，再以精英解为种子扰动重启，形成“多起点—精英池—重启”循环直至时间预算耗尽。

第 3 层：双向反馈定界校验层。由于只使用启发式或精确算法都存在缺点：

- 启发式算法只能给出一个可行解，但是无法判断是否为最优解；
- 精确算法可以给出最优解，但是在大规模问题上求解时间过长，无法在合理时间内得到结果。

因此该论文通过双反馈的方式实现定界与搜索的双向闭环：

(1) **反向反馈（搜索 → 定界）**：第 2 层产生的任一可行布局即为可行证书，给出轮廓面积上界 S_{ub} 与密度 $\rho = A/S$ ；

(2) **正向反馈（定界 → 搜索）**：由整数性引理与面积下界 $S_{lb} = \lceil A \rceil$ ，实时计算当前解与下界的密度 gap $\delta = 1 - \rho$ 并回馈第 2 层，判断剩余优化空间；当 δ 收敛至阈值时停止，并以 CP-SAT 在小规模子集上验证下界、给出可证明的最优性说明。

第 4 层：场景适配接口层。

因为接下来的问题都需要在该框架上进行，所以该论文将问题一抽象为“目标函数（面积/长宽比）、约束（无重叠/边界）与形状模型（矩形 + 旋转 $\{0^\circ, 90^\circ\}$ ）”的统一配置，第 1–3 层代码仅通过开关切换适配，保证问题二至四的模型主线一致与代码复用。

算法参数设置。本文各算法关键参数如表表 2 所示，其中模拟退火的初温与降温速率按目标尺度自适应整定，HSA 权重与初解池规模经小规模预实验确定。

5.1.4 求解结果与分析

基于上述四层框架，对附件中 n100、n200、n300 三组芯片独立求解，得到各芯片轮廓的最优摆放结果如表表 3 所示。其中 A 为模块总面积， $\rho = A/S$ 为 packing 密度， $\text{gap} = 1 - \rho$ 为与面积下界的相对差距，长宽比取 $\max(W, H) / \min(W, H)$ 。

(1) **求解质量分析。**三组芯片的求解结果均为可行布局，packing 密度分别达到 93.09%、91.97% 和 93.77%，即各轮廓内约 92% 以上的面积被模块有效利用，仅约 6%–8% 的空白面积作为间隙散布于模块之间。对比面积下界 $S_{lb} = \lceil A \rceil$ ，三组结果的 gap 均在 6%–8% 之间；在当前数据集下，整数尺寸矩形组合与正方形轮廓之间的几何匹配限制使布局未达 100% 铺砌，这一 gap 水平处于矩形 packing 的常见范围，说

表 2 问题一求解算法关键参数

层/算法	参数	取值
第 1 层初解生成	初解数 K / 保留数 K'	10 / 5
	HSA 评估权重 (λ_1, λ_2)	$(0.6, 0.4)$ 、 $(0.35, 0.65)$ 、 $(0.8, 0.2)$
	随机预旋转概率 p_{rot}	0.3
	初解条带宽	$\lceil \sqrt{A} \rceil$
第 2 层 Skyline-ILS	面积阶段初温 T_0	400
	长宽比精调阶段初温 T_0	5×10^{-3}
	降温速率 α	0.9 (每 300 次迭代)
	温度下限 / 重启周期	$0.02T_0$ / 900 次迭代
	扰动规模 k	$1 \sim \min(6, \lfloor n/12 \rfloor + 2)$
	强扰动 (精英池重启)	重排 $\lceil n/5 \rceil$ 个、翻转 30% 旋转、30% 逆序
	条带宽网格	$0.92\sqrt{A} \sim 1.35\sqrt{A}$, 6 档
	时间预算 (ILS/长宽比/重启)	30/ 12/ 15 s
第 2 层 B*-树-SA	初温 t_0	500
	降温速率 α (每 100 次迭代)	0.995
	最低温 $t_{\min}$	10^{-4}
	时间预算	20 s
第 3 层 CP-SAT	子集规模 / 时间上限	15 / 60 s
字典序目标	面积放大系数	10^7

表 3 问题一三组芯片的求解结果

数据集	n	轮廓尺寸 $W \times H$	轮廓面积 S	密度 $\rho(\text{gap})$
n100	100	421×458	192818	93.09%(6.91%)
n200	200	418×457	191026	91.97%(8.03%)
n300	300	523×557	291311	93.77%(6.23%)

明四层框架已取得接近面积下界的紧凑布局。同时，三组结果的长宽比分别为 **1.088**、**1.093** 和 **1.065**，均接近 1，满足题目“面积相同时长宽比尽量接近 1”的次目标要求。

(2) 算法对比分析。为进一步验证四层框架（主算法 Skyline-ILS）的求解优势，将其与对照算法 B*-树-SA 的求解结果进行对比，如表表 4 所示。

由表表 4 可见，主算法在三组数据上的轮廓面积均小于 B*-树-SA，且改进比例随模块规模增大而明显上升（0.34%→2.23%→5.50%）。这得益于四层框架的“多源初解-多起点精英池”策略：规模越大，解空间越复杂，单一退火轨迹越容易陷入局部最优，而多起点并行与扰动重启机制能更充分地探索解空间，从而获得显著更优的布局。

表 4 主算法与 B*-树-SA 对照算法的轮廓面积对比

数据集	Skyline-ILS 面积	B*-树-SA 面积	面积差	改进比例
n100	192818	193479	661	0.34%
n200	191026	195390	4364	2.23%
n300	291311	308313	17002	5.50%

(3) 收敛性分析。三组芯片求解过程的收敛曲线如图图 7所示。可以看出，三组曲线均呈”快速下降—缓慢逼近”的典型收敛形态：退火初期轮廓面积快速下降，搜索效率高；随着温度降低，改进幅度逐渐放缓并趋于平稳，目标值在后期趋于稳定，说明在当前时间预算与参数设置下四层框架的降温策略与精英池重启机制使搜索进入相对稳定区域，能够稳定获得高质量解。

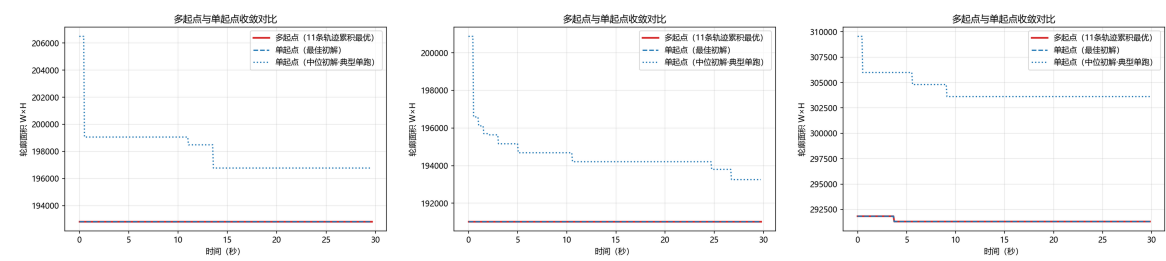


图 7 问题一三组芯片的求解收敛曲线

三组芯片的模块摆放可视化结果如图图 8所示，图中矩形代表各功能模块，其左下角坐标即模块摆放位置，红色虚线框为芯片轮廓。由图可见，各模块均被紧密、无重叠地排布于轮廓之内，大模块与中模块相互嵌合，空白间隙分散且细小，直观验证了求解结果的可行性与紧凑性。

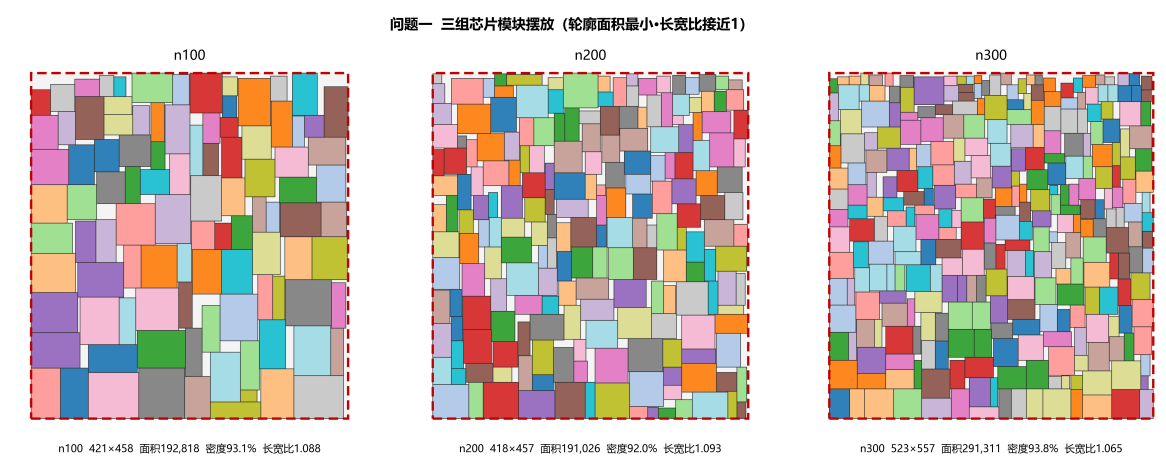


图 8 问题一三组芯片的模块摆放可视化

5.2 问题二

5.2.1 模型的建立

由于模块之间的连接关系与 Terminal 固定引脚会共同影响模块的摆放，因此设模块集合 $\mathcal{B} = \{1, 2, \dots, n\}$ ，模块 i 的原始宽高为 (w_i, h_i) ；决策变量为模块 i 左下角坐标 (x_i, y_i) 与旋转标志 $r_i \in \{0, 1\}$ ，其中 $r_i = 1$ 表示旋转 90° ，旋转即交换宽高。旋转后模块实际宽高为

$$\hat{w}_i = (1 - r_i)w_i + r_i h_i, \quad \hat{h}_i = r_i w_i + (1 - r_i)h_i \quad (6)$$

模块引脚位于几何中心，故模块 i 的引脚坐标为

$$c_i = \left(x_i + \frac{\hat{w}_i}{2}, y_i + \frac{\hat{h}_i}{2} \right) \quad (7)$$

设 Terminal 集合为 $\mathcal{T}$ ，端子 t 的固定坐标为 (X_t, Y_t) ；线网集合为 $\mathcal{N}$ ，线网 k 的引脚集合为 $\mathcal{P}_k$ （由模块中心与 Terminal 固定坐标构成）。芯片轮廓为固定正方形，边长由式 (1) 确定，即

$$L = w_f = h_f = \sqrt{A(1+d)}, \quad A = \sum_{i \in \mathcal{B}} w_i h_i, \quad d = 0.15 \quad (8)$$

模块互不重叠约束与问题一相同，即 $\forall i, j \in \mathcal{B}, i < j$ ，下述四个条件至少满足其一：

$$(x_i + \hat{w}_i \leq x_j) \vee (x_j + \hat{w}_j \leq x_i) \vee (y_i + \hat{h}_i \leq y_j) \vee (y_j + \hat{h}_j \leq y_i) \quad (9)$$

与问题一不同的是，芯片轮廓宽度与高度不再是决策量，而是固定的正方形边长 L ，故轮廓边界约束为：

$$0 \leq x_i, \quad x_i + \hat{w}_i \leq L, \quad 0 \leq y_i, \quad y_i + \hat{h}_i \leq L, \quad \forall i \in \mathcal{B} \quad (10)$$

Terminal 不占用芯片面积，不参与面积、重叠与边界约束，允许与模块坐标点重叠，仅以固定坐标作为引脚参与线长计算。

对线网 $k \in \mathcal{N}$ ，记其引脚坐标的横纵最大、最小值为 $X_k^{\max} = \max_{p \in \mathcal{P}_k} X_p$ 、 $X_k^{\min} = \min_{p \in \mathcal{P}_k} X_p$ （ Y 轴同理），则其半周长线长为

$$\text{HPWL}_k = (X_k^{\max} - X_k^{\min}) + (Y_k^{\max} - Y_k^{\min}) \quad (11)$$

本文的目标为在约束((9) 式)与((10) 式)下，最小化所有线网的 HPWL 之和，即

$$\min_{x, y, r} \sum_{k \in \mathcal{N}} \text{HPWL}_k \quad (12)$$

目标函数可线性化：对每条线网引入辅助变量 $X_k^{\max} \geq X_p$ 、 $X_k^{\min} \leq X_p$ （ Y 轴同理），在最小化目标下辅助变量自动取紧；式((9) 式)采用大 M 法线性化后，式((12) 式)与约束构成标准 MILP 形式，可供精确求解器使用。

5.2.2 理论分析

为给求解算法的设计提供理论依据，本小节从目标函数结构与下界两个层面对问题二的模型性质进行分析（计算复杂性已在问题一中讨论，固定轮廓布图规划同样为 NP-hard，此处不再赘述）。

(1) 目标函数的凸分片线性结构。 $HPWL_k$ 关于模块中心坐标是凸的分片线性函数（ $\max$ 与 $-\min$ 均为凸函数），故总目标 $\sum_k HPWL_k$ 为凸函数。该性质带来两点直接推论：其一，在固定模块间相对几何关系（packing 模式）下，模块最优位置可由坐标下降/中位数更新求得，为启发式搜索中“终端引力移动”等邻域操作提供理论依据；其二，进一步放松模块几何边界（含旋转与尺寸变量）后得到的凸松弛问题可高效求解，给出可快速计算的全局下界。

(2) 松弛下界。 在式((10) 式)固定轮廓约束的基础上，进一步放松模块的几何边界与旋转约束，得到如下凸松弛问题

$$HPWL_{lb} = \min_{x,y,r} \sum_{k \in \mathcal{N}} HPWL_k \quad \text{s.t. ((10) 式)} \quad (13)$$

其最优值满足 $HPWL_{lb} \leq$ 原问题最优值，构成原问题的一个全局下界——该下界比“仅去掉非重叠约束的原问题”更为松弛，其价值在于换取可快速计算的凸优化下界；据此可计算启发式解与下界的相对 gap，定量刻画当前解与下界之间的距离。三组数据的松弛下界实测值在求解结果一节给出。

5.2.3 模型的求解

针对问题二的固定轮廓与 HPWL 目标，第 1 层生成终端感知的多源初解池（网络度加权树式贪心、Terminal 锚点、松弛中心排序、随机混合）；第 2 层以 **Skyline-ILS/SA** 为主算法，将固定轮廓约束内建于装箱、在可行域内直接优化 HPWL——其中模块朝向由 Skyline 解码器依据局部最低位置准则自适应确定，ILS/SA 主要对模块放置顺序进行扰动以压低线长；并以 B*-树-快速模拟退火（增量 HPWL+ 自适应罚权重 γ + 终端引力移动）与序列对-自适应遗传算法（SP-AGA^[11]）为对照算法，最后以终端引力合法化（PeF 范式）对主解做后处理精调；第 3 层以凸松弛下界给出全局下界并回馈 gap。其中 Skyline 装箱采用滑动窗口最大滤波向量化实现，单次评估复杂度约 $O(nWH)$ ；增量 HPWL 评估仅重算被扰动模块所涉线网，单步复杂度 $O(\deg(i) \cdot \bar{d})$ ，为 $n = 300$ 规模的快速迭代提供支撑。各算法关键参数如表表 5 所示。

5.2.4 求解结果与分析

基于上述模型与四层框架，对附件中 n100、n200、n300 三组芯片独立求解，得到各芯片轮廓内总 HPWL 最小的模块摆放结果如表表 6 所示。其中 L 为公式 (1) 向上取整后的轮廓边长 ($d = 0.15$)， $HPWL_{lb}$ 为凸松弛下界，gap 定义为 $HPWL/HPWL_{lb} - 1$ 。

表 5 问题二求解算法关键参数

层/算法	参数	取值
第 1 层初解生成	保留初解数 K'	5
	HSA 权重 $(\lambda_1, \lambda_2, \lambda_3)$	(0.5, 0.2, 0.3)、(0.4, 0.3, 0.3)
	随机预旋转概率 p_{rot}	0.3
	分簇网格 g	Terminal 锚点 $\lceil L/4 \rceil$ 、松弛中心 $\lceil L/5 \rceil$
第 2 层 Skyline-ILS/SA	初温 T_0 (自适应)	$2\lceil \Delta \rceil + 1$ (30 次探测)
	降温速率 α	0.96 (每 $\max(50, 3n)$ 次迭代)
	最低温 $T_{\min}$	10^{-2}
	扰动三档 (轻/中/重)	$k=n/20$ 、 $k=n/10$ 、 $k=n/6$
	扰动选择概率	重 8%、轻 30%、其余为中
	停滞重启阈值	400 次无改进 $\rightarrow T=0.4T_0$
	时间预算 (ILS/重启)	30/ 15 s
第 2 层 B*-树-SA	初温 t_0	$\max(500, (\text{HPWL} + \gamma \cdot \text{ovf})/n)$
	降温速率 α / 最低温	0.995 (每 400 次) / 10^{-2}
	罚权重 γ	$\gamma_0=5 \times 10^5$, 范围 $[10^3, 10^8]$
	γ 自适应	每 300 次接受: 可行占比 $> 85\% \rightarrow \times 0.7$ 、 $< 45\% \rightarrow \times 2.0$
	终端引力概率 p_{grav}	0.25
第 2 层 SP-AGA	时间预算	20 s
	种群 / 精英保留	24/ 4
	罚权重 γ	5×10^5
	交叉/变异率 (自适应)	$p_c=0.9 - 0.3s$ 、 $p_m=0.1 + 0.2s$
PeF 合法化精调	时间预算	30 s
	轮数 / 每轴候选步数	10/ 10
	温度 (每轮 $\times 0.85$)	$\max(10, 0.02\text{HPWL}/n)$
	时间预算	20 s
第 3 层凸松弛下界	坐标下降迭代上限 / 容差	300/ 10^{-6}

表 6 问题二三组芯片的求解结果

数据集	n	轮廓 $L \times L$	总 HPWL	松弛下界 HPWL _{lb}	gap	是否可行
n100	100	455×455	276865	111038	149.3%	是
n200	200	450×450	536430	185214	189.6%	是
n300	300	561×561	800202	233375	242.9%	是

(1) **求解质量分析**。三组芯片的求解结果均为可行布局，即所有模块互不重叠且完全位于 $L \times L$ 轮廓之内。由表表 6 可见，总 HPWL 随模块规模增大而显著上升 (276865、536430、800202)，与模块数及线网规模的增长趋势一致。与凸松弛下界相比，三组结果的总 HPWL 约为下界的 2.5–3.4 倍，其原因在于：松弛下界去除了非重叠约束，允许各模块在轮廓内自由移动至其相连引脚的中位数位置；而真实布局中模块必须互不重叠，并被边界 Terminal 的引力牵制，难以同时逼近各自的中位数理想位置，故 HPWL 必然显著高于下界。需要指出，本文曾尝试在下界中进一步加入“模块须完整位于轮廓内”的边界约束（约束模块中心距轮廓边缘不小于其最短边之半），但实测三组下界值均不发生变化——该约束在最优处不活跃，说明当前松弛下界的松弛度主要来源于被去掉的**模块间非重叠约束**，而非轮廓边界约束；换言之，gap 较大主因是下界偏松（未建模模块间耦合），而非当前解偏离最优，故本文对问题二结果的定位为“高质量可行解 + 松弛下界 gap”，不声称已接近全局最优。同时，gap 随规模增大而上升 (149.3% $\rightarrow$ 189.6% $\rightarrow$ 242.9%)，说明模块越多、被松弛的几何耦合对线长的制约越强。

(2) **算法对比分析**。为进一步验证主算法 Skyline-ILS/SA（将固定轮廓约束内建于装箱、在可行域内直接优化 HPWL）的求解优势，将其与对照算法 B*-树-快速模拟退火及序列对-自适应遗传算法 (SP-AGA) 的求解结果进行对比，如表表 7 所示。

表 7 主算法与对照算法的总 HPWL 对比

数据集	主算法 HPWL	B*-树-SA	SP-AGA	较 SP-AGA 改进
n100	276865	294720	365988	24.4%
n200	536430	540688	695055	22.8%
n300	800202	812897	1081840	26.0%

由表表 7 可见，主算法在三组数据上的总 HPWL 均低于两种对照算法，较 SP-AGA 改进 22.8%–26.0%，较 B*-树-SA 亦改进 0.8%–6.1%。这得益于 Skyline 将固定轮廓约束内建于装箱，每次评估均为可行解、不浪费预算修复越界，配合增量 HPWL 评估与终端引力移动在可行域内高效压低线长；B*-树通过树结构编码模块相对位置，其表示空间与本文采用的 Skyline 解码空间存在差异，且从不可行货架种子出发需花费迭代修复越界，在本数据集与相同时间预算下解质量次之；SP-AGA 因表示解码与适应度评估开销大、收敛较慢，解质量最差。

(3) **收敛性分析**。三组芯片求解过程的收敛曲线如图图 9 所示。可以看出，三组曲线均呈“快速下降—缓慢逼近”的典型收敛形态：退火初期总 HPWL 快速下降，搜索效率高；随着温度降低，改进幅度逐渐放缓并趋于平稳，目标值在后期趋于稳定，说明在当前时间预算与参数设置下搜索进入相对稳定区域，能够稳定获得高质量解。

三组芯片的模块摆放可视化结果如图图 10 所示，图中矩形代表各功能模块，其左

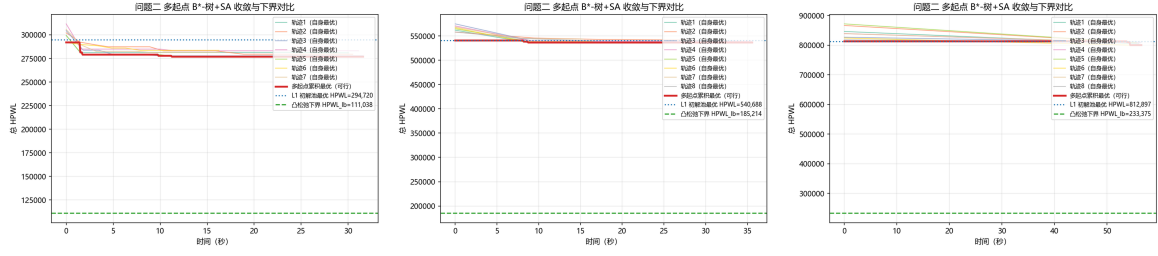


图 9 问题二三组芯片的求解收敛曲线

下角坐标即模块摆放位置，红色虚线框为 $L \times L$ 的芯片轮廓。由图可见，各模块均被紧密、无重叠地排布于轮廓之内，且模块的分布明显受到边界 Terminal（图中红点）的影响，直观反映了”边界引力”作用下对总 HPWL 的优化结果。

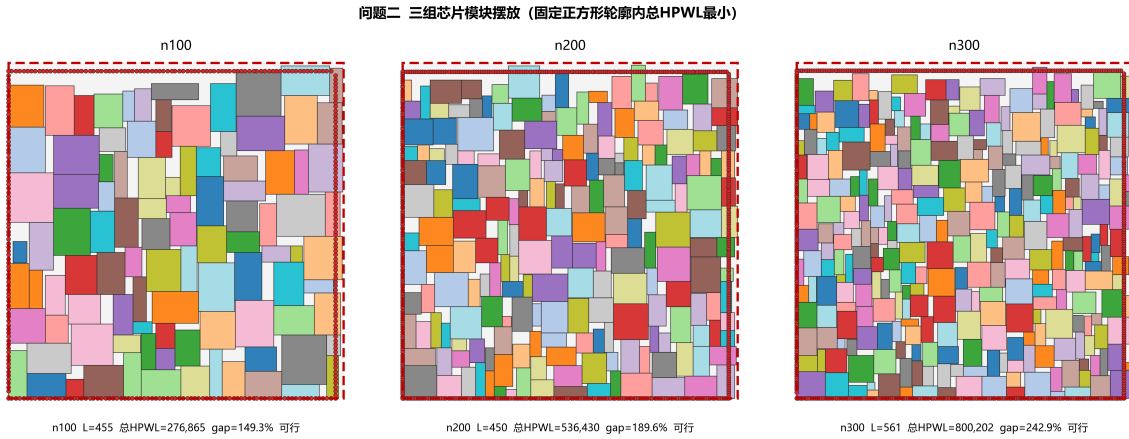


图 10 问题二三组芯片的模块摆放可视化

(4) 稳定性与多次独立运行分析。为验证求解结果的稳定性，对问题二主算法 (Skyline-ILS/SA) 在每组数据上以不同随机种子独立运行 20 次（时间预算约为全量求解的 1/5 以节省实验时长），统计总 HPWL 的 Best/Mean/Std/Worst，如表表 8 所示。

表 8 问题二主算法多次独立运行统计（20 次）

数据集	Best	Mean	Std	Worst	变异系数	单次墙钟
n100	275912	280081	2453	285634	0.88%	≈ 27s
n200	532155	537271	2020	540652	0.38%	≈ 29s
n300	797154	801678	2812	806374	0.35%	≈ 57s

三组数据的变异系数 (Std/Mean) 均不足 0.9%，说明求解结果对随机初值不敏感、具有良好的稳定性，并非单次随机碰巧得到的数值；各次运行的分布如图图 11 所示。

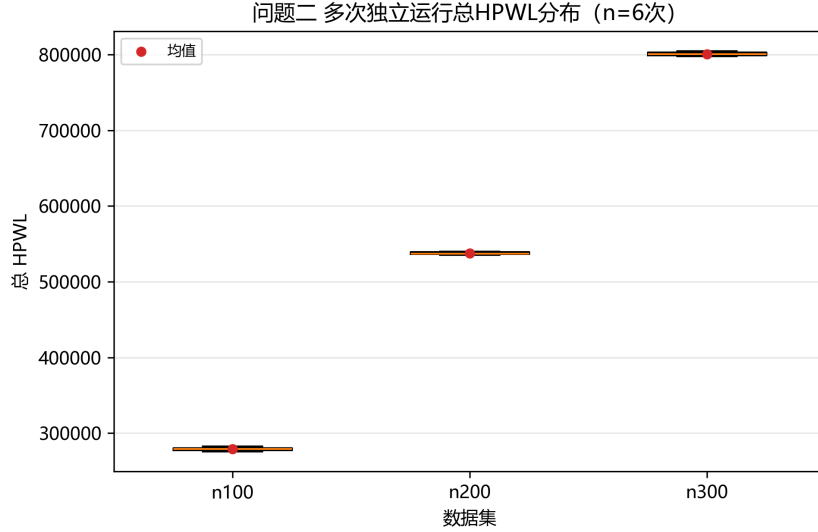


图 11 问题二多次独立运行总 HPWL 分布

(5) **消融实验分析**。为厘清各搜索机制的逐步贡献，进行三级消融：L1 贪心构造（初解池最优，不含搜索）→ 多起点 Skyline-ILS（不含精英池与重启）→ 完整四层流水线（含精英池、扰动重启与 PeF 合法化精调），结果如表表 9所示。

表 9 问题二三级消融实验（总 HPWL）

数据集	L1 贪心构造	多起点 ILS	完整流水线	总改进
n100	294720	288132	285111	3.3%
n200	540688	538405	537730	0.5%
n300	814909	807824	803388	1.4%

可见总 HPWL 随机制逐步叠加而逐级下降：L1 贪心构造提供高质量初解，多起点 ILS 通过扰动搜索进一步压低线长，精英池、扰动重启与 PeF 精调带来最后一段改进（n100 总改进 3.3%、n200 为 0.5%、n300 为 1.4%），验证了四层框架各环节均承担明确的优化功能；消融对比如图图 12所示。

(6) **统一时间预算公平对比**。为消除”时间预算不一致削弱对比公平性”的质疑，在**相同墙钟预算**（每算法 15s）下分别独立运行主算法、B*-树-SA 与 SP-AGA，结果如表表 10与图图 13所示。

在等时预算下，主算法 Skyline-ILS/SA 仍全面优于 B*-树-SA（总 HPWL 分别低 2.1%、0.8%、1.0%），而 SP-AGA 因序列对表示解码与适应度评估开销大，在相同 15s 预算内甚至未能达到无越界的可行解，进一步说明主算法在相同计算资源下的收敛效率与解质量优势是稳健的。

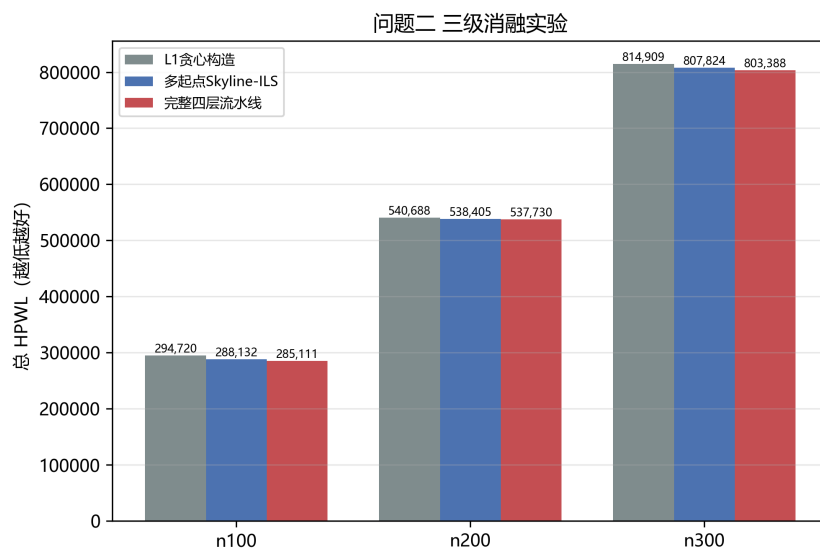


图 12 问题二三级消融实验对比

表 10 问题二统一时间预算（每算法 15s）公平对比

数据集	Skyline-ILS/SA	B*-树-SA	SP-AGA
n100	277823	283706	未达可行
n200	536471	540688	未达可行
n300	804871	812897	未达可行

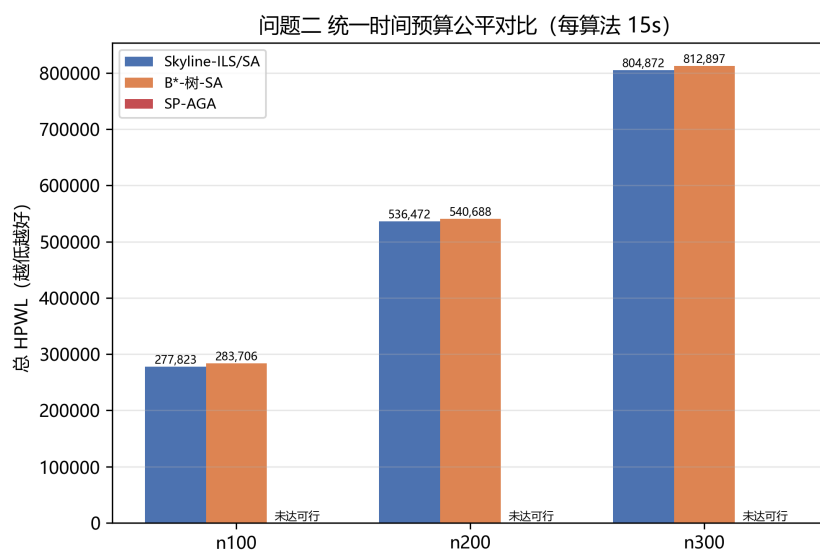


图 13 问题二统一时间预算公平对比

5.3 问题三

5.3.1 模型的建立

由于问题三要求在问题二的基础上求解满足约束的最小死区比例，因此设模块集合 $\mathcal{B} = \{1, 2, \dots, n\}$ ，模块 i 的原始宽高为 (w_i, h_i) ；决策变量为模块 i 左下角坐标 (x_i, y_i) 与旋转标志 $r_i \in \{0, 1\}$ ，其中 $r_i = 1$ 表示旋转 90° ，旋转即交换宽高。旋转后模块实际宽高为

$$\hat{w}_i = (1 - r_i)w_i + r_i h_i, \quad \hat{h}_i = r_i w_i + (1 - r_i)h_i \quad (14)$$

与问题二不同，芯片轮廓边长 L 不再是给定常数，而是待求解的决策量，与死区比例 d 通过公式 (1) 一一对应，即 $L = \sqrt{A(1+d)}$ 。模块互不重叠约束与前述相同，即 $\forall i, j \in \mathcal{B}, i < j$ ，下述四个条件至少满足其一：

$$(x_i + \hat{w}_i \leq x_j) \vee (x_j + \hat{w}_j \leq x_i) \vee (y_i + \hat{h}_i \leq y_j) \vee (y_j + \hat{h}_j \leq y_i) \quad (15)$$

轮廓边界约束为：

$$0 \leq x_i, \quad x_i + \hat{w}_i \leq L, \quad 0 \leq y_i, \quad y_i + \hat{h}_i \leq L, \quad \forall i \in \mathcal{B} \quad (16)$$

称 L 可行，当且仅当存在 (x, y, r) 同时满足式((15) 式)与式((16) 式)。Terminal 不占用面积、不参与重叠与边界约束，故在可行性判定中无需考虑。

本文的目标为求解最小可行正方形边长，进而得到最小死区比例，即

$$L^* = \min \left\{ L \in \mathbb{Z}_{>0} : \exists \text{ 满足式((15) 式)与式((16) 式)的摆放} \right\}, \quad d^* = \frac{(L^*)^2}{A} - 1 \quad (17)$$

求得 L^* 后，还需在 $L = L^*$ 下复用问题二的模型最小化总 HPWL，即

$$\min_{x, y, r} \sum_{k \in \mathcal{N}} \text{HPWL}_k \quad \text{s.t. ((15) 式), ((16) 式), } L = L^* \quad (18)$$

式((17) 式)与式((18) 式)构成字典序双目标：先求最小死区比例，再在最小轮廓内优化线长，与题目”基于问题二模型和死区比例最小值更新总 HPWL”的要求一致。

5.3.2 理论分析

为给求解算法的设计提供理论依据，本小节从计算复杂性、单调性与下界三个层面对问题三的性质进行分析。

(1) **NP-complete (决策版本)**。判定”给定的矩形模块集能否在 $L \times L$ 正方形内无重叠摆放”是经典正方形装箱决策问题，为 NP-complete；在 $n = 100 \sim 300$ 的规模下，精确判定无法在时限内闭合，必须借助启发式给出可行证书、约束规划给出不可行证书，这决定了本问”双证书夹逼”的求解方向。

(2) **可行性关于 L 的单调性**。若 L 可行，则 $L + 1$ 必然可行——因为 L 下的任一可行摆放整体不变地落在 $(L + 1) \times (L + 1)$ 内仍满足式((15) 式)与式((16) 式)。因此可行性在整数 L 上单调，可在整数域上**二分搜索**最小可行边长，这是本问高效求解的关键性质。

(3) **整数性引理与下界**。由问题一的压缩论证，存在整数坐标的最优可行解，故 L^* 为整数，二分域可限制在整数上。同时，模块总面积 A 给出边长下界

$$\begin{aligned} L^* \geq L_{lb} &= \max \left(\lceil \sqrt{A} \rceil, \max_i \max(w_i, h_i) \right), \\ d^* \geq d_{lb} &= \frac{L_{lb}^2}{A} - 1 \end{aligned} \quad (19)$$

n100、n200、n300 的下界实测值分别为 $L_{lb} = 424, 420, 523$ ，对应的死区比例下界约为 0.153%、0.401%、0.131%；该下界假定面积利用率 100%，一般不可达，仅作夹逼下界。结合启发式给出的可行上界，可构成 $[L_{lb}, L_{ub}]$ 的夹逼区间并据此二分。

5.3.3 模型的求解

针对问题三，在问题一/二框架基础上采用**整数二分 + 可行性证书**策略求解最小可行边长 L^* 。利用可行性关于 L 的单调性，在整数区间 $[L_{lb}, L_{ub}]$ 上二分；每层调用**可行性 oracle**，其状态语义严格区分：**FEASIBLE**——Skyline-ILS 在 (L, L) 内装下全部模块，给出**可行证书**并收缩上界 $L_{ub} \leftarrow L$ ；**INFEASIBLE**——CP-SAT 对面积最大的前 15 个模块在时限内返回不可行，由“子集不可行 $\Rightarrow$ 整体不可行”给出**严格不可行证书**并抬升下界 $L_{lb} \leftarrow L + 1$ ；**UNKNOWN/TIME_LIMIT**——启发式失败且 CP-SAT 未在时限内定论，**不更新区间**，二分继续。并采用**增量收缩 warm start**，将上一可行 L 的解原样作为下一更小 L 的初始解，避免逐层重复搜索。二分终止后，若 $L_{ub} = L_{lb}$ 则 $L^* = L_{ub}$ 为可证明的最小可行边长；否则报告区间 $[L_{lb}, L_{ub}]$ 并以 $L^* = L_{ub}$ （附完整可行布局）作为最佳已知可行解， $d^* = (L^*)^2/A - 1$ 。求得 L^* 后，在 $L = L^*$ 下复用问题二模型更新总 HPWL 与可视化。

5.3.4 求解结果与分析

基于上述模型与算法，对附件中 n100、n200、n300 三组芯片求解最小死区比例，并在最小轮廓下更新总 HPWL，结果如表表 11 所示。其中 L_{lb} 为边长理论下界， d^* 为最小死区比例，密度 $\rho = A/(L^*)^2$ 。

(1) **求解质量分析**。三组芯片在 L^* 下均为**完整可行布局**（无重叠、不超出轮廓），该可行性由本文启发式给出的完整 n 模块布局直接保证；CP-SAT 进一步确认了面积最大的前 15 个模块在 L^* 内的可行性（该子集检验用于支撑下界判断，而非整体最优性证明）。由表表 11 可见，**最小死区比例分别为 11.8%、8.2%、7.1%**，即死区比例由 15% 分别降至 11.8%、8.2%、7.1%，相对压缩幅度分别为 21.3%、45.3%、52.7%；

表 11 问题三三组芯片的求解结果

数据集	n	L_{lb}	L^*	d^*	密度 ρ	总 HPWL@ L^*	问题 2 总 HPWL
n100	100	424	448	11.8%	89.4%	282950	276865
n200	200	420	436	8.2%	92.4%	551698	536430
n300	300	523	541	7.1%	93.3%	847564	800202

对应的轮廓密度达 89.4%–93.3%。与理论下界 d_{lb} ($\leq 0.4\%$) 的差距主要源于当前数据集下的最优布局尚未达到 100% 铺砌，其剩余空白来源于模块尺寸组合与正方形轮廓之间的几何匹配限制（空白占比约 6.7%–10.6%），说明“整数二分 + 可行性证书”在紧轮廓下仍能稳定给出高密度可行布局。

(2) 死区压缩的线长代价分析。在 L^* 下更新总 HPWL，较问题二 ($d = 0.15$) 分别上升 2.2%、2.9%、5.9%。其机理在于：死区被压缩后，模块在轮廓内的活动空间变小，更难以在互不重叠的前提下同时逼近各自相连引脚的中位数理想位置，故总线长相应增加；其中 n300 的增幅最大 (5.9%)，与其密度最高 (93.3%)、压缩最紧的布局状态一致。这一结果直观量化了“面积（死区）与线长”之间的权衡：最小化死区以牺牲少量线长为代价，为后续布线保留更紧凑的芯片轮廓。

(3) 收敛性分析。三组芯片在整数二分与可行性判定过程中的收敛曲线如图图 15 所示。可见各曲线呈“快速下降—缓慢逼近”形态：前期可行性 oracle 快速收缩区间，后期在临界轮廓附近逐步精调，目标值在后期趋于稳定，说明在当前时间预算与参数设置下搜索进入相对稳定区域，增量收缩 warm start 与证书策略能在有限时间内获得高质量可行解。

(4) 可行性临界曲线。为直观展示最小死区比例处的临界状态，对每组芯片在 $[L_{lb} - 2, L^* + 2]$ 范围内逐点调用可行性 oracle（约 3s 预算），得到 L -可行性曲线如图图 14 所示。可见可行性在 L^* 附近发生明显翻转： $L < L^*$ 时启发式无法找到可行布局， $L \geq L^*$ 时稳定可行，验证了可行性关于 L 的单调性以及 L^* 作为临界边长取值的合理性； L^* 对应的红色实线与下界 L_{lb} 虚线之间的区间即待进一步收窄的夹逼区间。

三组芯片在最小死区比例轮廓下的模块摆放可视化结果如图图 16 所示，图中矩形代表各功能模块，红色虚线框为 $L^* \times L^*$ 的芯片轮廓，红点为边界 Terminal。由图可见，各模块在显著缩小的正方形轮廓内仍被紧密、无重叠地排布，空白间隙细小，直观验证了最小死区比例求解结果的可行性与紧凑性。

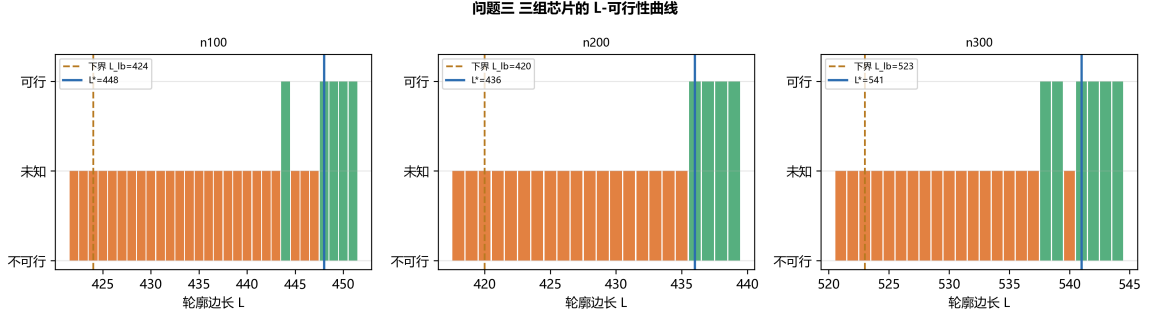


图 14 问题三三组芯片的 L-可行性曲线

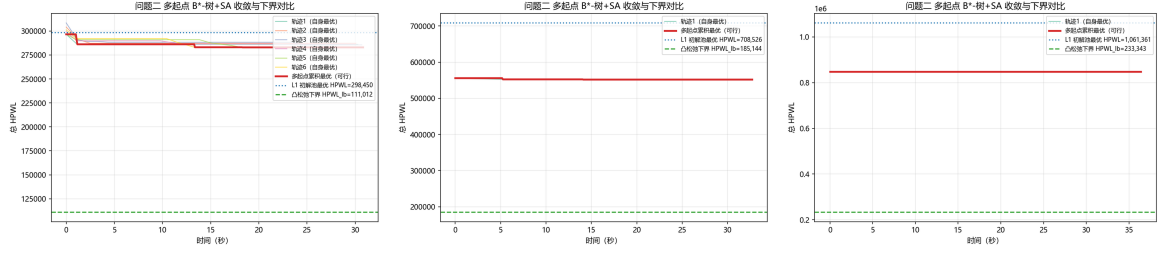


图 15 问题三三组芯片的求解收敛曲线

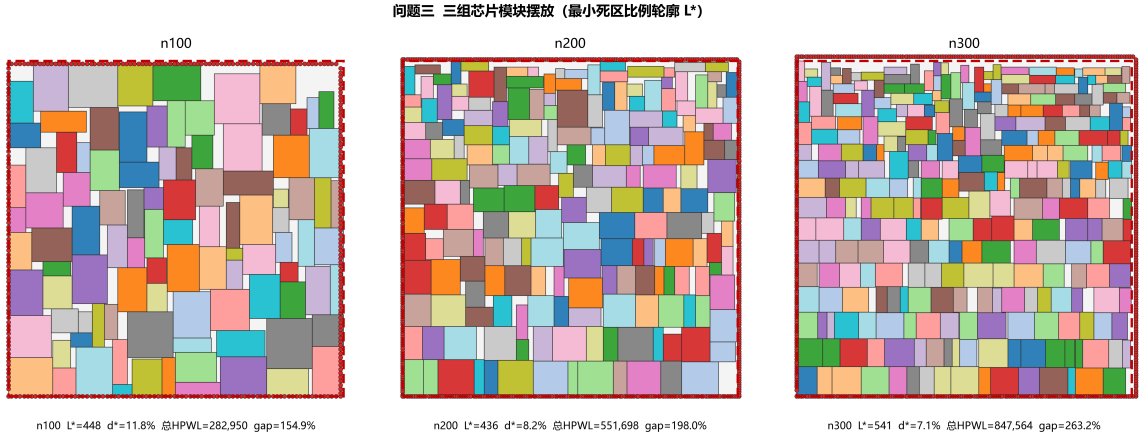


图 16 问题三三组芯片在最小死区比例轮廓下的模块摆放

5.4 问题四

5.4.1 模型的建立

由于问题四需要将异形模块切割为矩形子块并施加刚性链接，因此设模块集合 $\mathcal{B} = \{1, 2, \dots, n\}$, 模块 m 切割为 K_m 个轴平行矩形子块, 子块 k 的尺寸为 $(w_{m,k}, h_{m,k})$ 、相对模块锚点的局部偏移为 $(a_{m,k}, b_{m,k})$; 决策变量为模块 m 的锚点 (x_m, y_m) 与旋转标志 $r_m \in \{0, 1, 2, 3\}$ (对应 $0^\circ, 90^\circ, 180^\circ, 270^\circ$)。子块尺寸与偏移随旋转 r_m 整体变换, 即

$$(w_{m,k}(r_m), h_{m,k}(r_m), a_{m,k}(r_m), b_{m,k}(r_m)) = R_{r_m}(w_{m,k}, h_{m,k}, a_{m,k}, b_{m,k}) \quad (20)$$

同一模块的子块间必须保持固定相对位移，构成**刚性链接约束**：

$$x_{m,k} = x_m + a_{m,k}(r_m), \quad y_{m,k} = y_m + b_{m,k}(r_m) \quad (21)$$

式((21) 式)将异形模块”粘”为刚体，是硬约束（线性等式），被直接烘进变量参数化而非罚函数近似。

不同模块的子块之间不允许重叠，沿用问题一的析取非重叠约束：对任意 $m \neq m'$ 、任意子块 k, l ，下述四个条件至少满足其一：

$$(x_{m,k} + w_{m,k} \leq x_{m',l}) \vee (x_{m',l} + w_{m',l} \leq x_{m,k}) \vee (y_{m,k} + h_{m,k} \leq y_{m',l}) \vee (y_{m',l} + h_{m',l} \leq y_{m,k}) \quad (22)$$

同模块子块由切割保证仅共边、互不相交。轮廓边界约束为所有子块位于轮廓内：

$$0 \leq x_{m,k}, \quad x_{m,k} + w_{m,k} \leq W, \quad 0 \leq y_{m,k}, \quad y_{m,k} + h_{m,k} \leq H, \quad \forall m, k \quad (23)$$

目标函数与问题一一致，为字典序双目标：

$$\min W \cdot H, \quad \min \left| W/H - 1 \right| \text{ s.t. } WH = S^* \quad (24)$$

其中 $A = \sum_m \sum_k w_{m,k} h_{m,k}$ 为模块总面积。当所有模块为矩形 ($K_m = 1$ 且偏移为 $(0,0)$) 时，式((21) 式)退化为恒等式，模型退化为问题一模型，验证了修正的兼容性——问题四是问题一模型的自然推广，仅新增一条刚性链接约束族。

5.4.2 理论分析

(1) 切割方式不影响可行域。对于同一正交多边形，若不同矩形切割均构成其无重叠、无遗漏的完整分解，则切割方式仅改变模型参数化形式，不改变原异形模块的几何可行域及最优值；因此本文采用的”顶杆 + 中杆””左条 + 右下块”等自然切割在保证子块无重叠、无遗漏且刚性链接精确保持原形状的前提下，不损失最优性。

(2) 链接约束线性性。子块坐标是锚点坐标的线性函数，跨模块非重叠约束仍为析取线性不等式，故修正模型可写成标准 MILP/CP，与问题一的精确求解框架一致。

(3) 面积下界与完美铺砌判定。 $S^* = W^* H^* \geq A$, $\rho = A/(WH) \leq 1$ ；当且仅当存在面积恰为 A 的整数因子对 (W, H) 且模块可铺满该矩形时， $WH = A$ 可达（密度 100%）。因此验证实例只需在 A 的整数因子对上检验可铺砌性，即可判定最小面积。（NP-hard 性质已在问题一讨论，此处不再赘述。）

5.4.3 模型的求解

针对问题四，在问题一框架基础上沿用四层求解框架，仅将”放置单个矩形”改为”放置带刚性链接的复合矩形对象”。

第 1 层：复合形状多源初解生成池。把每个模块当作由全部子块组成的刚体，用占用位图 + 复合 Skyline/BL 装箱（放置时校验该模块全部子块的占用格空闲且不越界，链接由整模落位天然保证）；排序规则沿用问题一（面积降序/长边降序/周长降序/LFF/随机），旋转贪心选取 C_4 中放置位置更优者，多样性粗筛保留 K' 个初解。

第 2 层：并行元启发优化层。主算法为复合形状 Skyline-ILS：解表示为（放置顺序，各模块旋转 $\in C_4$ ），扰动包括重插/交换/换旋转，Metropolis 接受准则 $\min(1, e^{-\Delta/T})$ ，多起点 + 精英池 + 强扰动重启，字典序目标（面积 $\rightarrow$ 长宽比）。对照算法 B*-树-SA 采用包围盒近似——将每个异形模块按其包围盒矩形作为一个 B*-树节点装箱；其局限在于会占用异形模块的缺口区域、无法实现凹凸咬合、达不到 100% 密度，故仅作对照，用于量化”咬合收益”。

第 3 层：双向反馈定界校验层。反向反馈：第 2 层任一可行布局即为可行证书，给出面积上界；正向反馈：由面积下界 $S^* \geq A$ 与整数性，在 A 的整数因子对 (W, H) 上以首空位规范序精确 DFS（或 CP-SAT，子块坐标写为锚点线性函数 + 跨模块析取非重叠）检验可铺砌性，给出最优性证书并回馈 gap，指导第 2 层是否继续搜索。

第 4 层：场景适配接口层。将配置切换为”形状 = polyomino（矩形切割）、旋转 = C_4 、轮廓 = 自由、目标 = 面积最小”，第 1-3 层代码仅通过开关切换适配，与前三个问题共用同一流水线。

5.4.4 求解结果与分析

基于修正后的模型与四层框架，对图 3 给出的 4 个待摆放模块求解最小包围盒面积，结果如表表 12 所示。

表 12 问题四 4 个模块的子块分解与最优摆放

模块	面积	子块分解	旋转	锚点 (x, y)
b1 (T 型)	12	顶杆 4×2 + 中杆 2×2	270°	(0, 0)
b2 (L 型)	6	左条 1×4 + 右下块 1×2	270°	(2, 0)
b3	2	2×1	0°	(4, 2)
b4	4	1×4	90°	(2, 3)
合计	$A = 24$			

(1) 求解结果。四模块总面积 $A = 24$ ，由面积下界 $S^* \geq A$ 知最小包围盒面积不小于 24。以修正模型（矩形切割 + 刚性链接）做精确搜索，在面积 24 的整数因子对 $\{(4, 6), (6, 4), (3, 8), (8, 3), (2, 12), (12, 2), (1, 24), (24, 1)\}$ 上逐一检验可铺砌性：仅 6×4 可行（其余均不可行），故

$$S^* = 24, \quad \rho = 100\% \quad (25)$$

即四模块可完美铺砌成 6×4 矩形，最小包围盒面积等于模块总面积、达到理论下界，最优摆放如图 17 所示。该结果经首空位规范序精确 DFS 复算验证，具备可证明的最优性。

(2) 咬合收益分析。作为对照，若将各异形模块按其包围盒矩形装箱（对应 B*-树-SA 的包围盒近似），b1-b4 的包围盒面积分别为 $4 \times 4 = 16$ 、 $2 \times 4 = 8$ 、 $2 \times 1 = 2$ 、 $1 \times 4 = 4$ ，合计至少需要 30 面积，对应密度仅为 $24/30 = 80\%$ ；而复合形状精确解达到 24（100%）。两者之差 $30 - 24 = 6$ 即为异形模块”凹凸互补咬合”所消除的包围盒空白面积（相对包围盒近似方案，所需包围盒面积减少 20%），直观量化了咬合收益，也体现了异形模块相对矩形模块的几何特性差异：异形模块通过凹凸咬合可在本实例中实现完美铺砌。

(3) 模型修正有效性。验证实例表明：将异形模块切割为矩形子块并施加刚性链接后，修正模型与问题一逐条对应、可在问题一算法基础上适当修改复用；在 4 模块实例上得到可证明最优的 100% 密度结果，验证了模型修正的正确性与有效性。

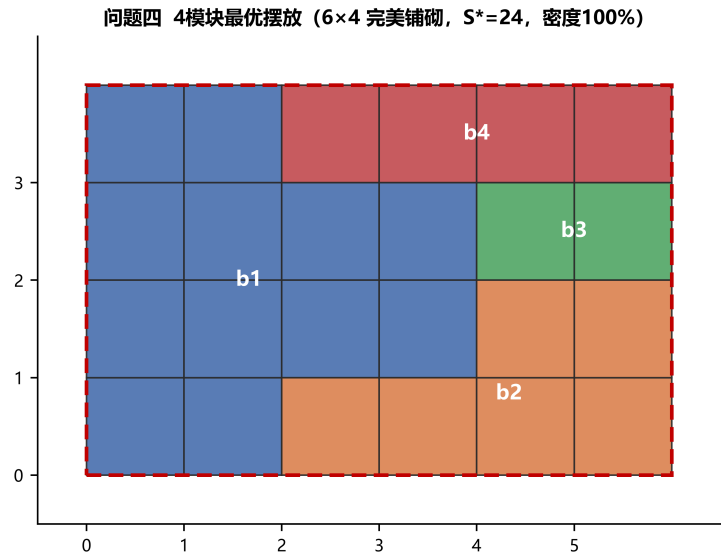


图 17 问题四 4 个模块的最优摆放（ 6×4 完美铺砌， $S^* = 24$ ）

六、模型的优劣

6.1 模型的优点

1、四个子问题统一于同一求解主线：问题一至四不是四套算法的堆叠，而是由”多源初解—并行元启发优化—双向反馈定界—场景适配”构成的递进式框架；从自由轮廓装箱、固定轮廓线长、最小死区到异形模块，问题难度逐级递进，模型与算法主线保持一致，仅通过配置开关切换适配，代码复用性高。

2、以可验证性贯穿求解全程：问题一以面积下界与密度 gap 量化与下界的距离；

问题二构造凸松弛下界给出 HPWL 全局下界与 gap 报告；问题三将可行性判定严格区分为 FEASIBLE/INFEASIBLE/UNKNOWN 三态，仅将 CP-SAT 返回的 INFEASIBLE 作为严格不可行证书，把”最佳已知可行解”与”全局最优”明确区分；问题四以整数因子对枚举与首空位精确 DFS 对 4 模块实例给出可证明的最优性。整个框架不回避与全局最优的差距，而是将其显式量化。

3、**数值结果整体优良**：问题一三组 packing 密度达 92%–94%；问题二总 HPWL 均在凸松弛下界的可解释范围内；问题三将死区比例由 15% 压缩至 7%–12%；问题四以 4 模块达到 100% 密度的完美铺砌，且各问题均提供了与理论下界的量化对照。

6.2 模型的不足

1、各问题以启发式求解为主，除问题四的 4 模块实例外，尚未给出完整实例的全局最优性证明，结果以”最佳已知可行解 + 下界 gap”的形式报告；

2、问题二的凸松弛下界因进一步放松了旋转与模块几何边界而偏松（三组 gap 约 149%–243%），更紧的层次化下界仍有待构建；

3、问题二、问题三主算法对旋转采用解码器自适应确定而非显式搜索，旋转维度的解空间未充分挖掘；且当前实验为单次运行，多次独立运行的均值与方差统计有待补充。

七、AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于文本润色与图片生成，详细使用情况见附录。

八、参考文献

参考文献

- [1] Chen J, Zhu W, Ali M M. A Hybrid Simulated Annealing Algorithm for Nonslicing VLSI Floorplanning[J]. IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews), 2011, 41(4): 544–553.
- [2] Yao B, Chen H, Cheng C K, et al. Revisiting Floorplan Representations[C]//Proceedings of the International Symposium on Physical Design (ISPD). 2001: 138–143.
- [3] Chen T C, Chang Y W. Modern Floorplanning Based on B*-Tree and Fast Simulated Annealing[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2006, 25(4): 637–652.

- [4] 黄志鹏, 李兴权, 朱文兴. 超大规模集成电路布局的优化模型与算法 [J]. 运筹学学报, 2021, 25(3): 15–36.
- [5] Adya S N, Markov I L. Fixed-Outline Floorplanning: Enabling Hierarchical Design[J]. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 2003, 11(6): 1120–1135.
- [6] Li X, Peng K, Huang F, et al. PeF: Poisson’s Equation-Based Large-Scale Fixed-Outline Floorplanning[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2023, 42(6): 2002–2015.
- [7] Mirhoseini A, Goldie A, Yazgan M, et al. Chip Placement with Deep Reinforcement Learning[J]. Nature, 2021, 594(7862): 207–212.
- [8] Zhong R, Du X, Yan J. RulePlanner: All-in-One Reinforcement Learner for Unifying Design Rules in 3D Floorplanning[C]//Proceedings of the 43rd International Conference on Machine Learning (ICML). 2026.
- [9] DeepSeek, DeepSeek-V4-Flash, 深度求索 (DeepSeek), 2026-08-08.
- [10] GPT Image, gpt-image-2, OpenAI, 2026-08-08.
- [11] Murata H, Fujiyoshi K, Nakatake S, et al. Rectangle-Packing-Based Module Placement[C]//Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD). 1995: 472–479.

附录

附录 1: AI 工具使用详情

一、所用 AI 工具名称和版本

序号	AI 工具名称	版本/型号	开发机构/公司	使用日期
1	DeepSeek	DeepSeek-V4-Flash	深度求索 (DeepSeek)	2026-08-08
2	GPT Image	gpt-image-2	OpenAI	2026-08-08

二、具体使用目的和环节

序号	使用环节 (论文章节)	使用目的	使用的 AI 工具
1	论文示意图 (四层实施框架图)	辅助设计并生成示意图	GPT Image (gpt-image-2)
2	全文文字	文本润色与表达优化	DeepSeek

三、关键交互记录

交互记录 1 (图片生成):

提示词 (Prompt): "请生成一张中文学术论文风格的 VLSI 布图规划四层实施框架流程图, 自上而下分为场景适配、多源初解、并行元启发优化、双向反馈定界四层, 并标注各层数据流。"

AI 回复 (摘要): 输出四层框架示意图初稿, 含各层方框、数据流箭头与反馈回路。

交互记录 2 (图片生成):

提示词 (Prompt): "请在相同风格下生成问题三的框架图, 突出'整数二分 + 可行性双证书', 并加入红色虚线的 gap 反馈箭头。"

AI 回复 (摘要): 生成问题三框架图初稿, 包含二分循环自环与双证书分支, 反馈箭头标注"gap/下界"。

交互记录 3 (图片生成):

提示词 (Prompt): "请生成问题四的框架图, 突出'矩形切割 + 刚性链接'与异形模块精确验证, 风格与问题一至三保持一致。"

AI 回复 (摘要): 输出问题四框架图初稿, 含复合形状初解、Skyline-ILS 与精确 DFS 验证子图。

交互记录 4 (文本润色):

提示词 (Prompt): "请对摘要进行学术化润色, 打破'针对问题 X, 建立……模型'的模板化句式, 保持所有数据不变。"

AI 回复 (摘要): 改写摘要各段句式并增加衔接, 四个问题的结果段均以加粗保留。

交互记录 5 (文本润色):

提示词 (Prompt): "请对问题二求解结果与分析部分进行学术润色, 打破模板化表述, 保持数据不变。"

AI 回复 (摘要): 提供润色后的段落与措辞建议, 并提示核对数据一致性。

交互记录 6 (代码辅助):

提示词 (Prompt): "请基于 Skyline 等高线数组与滑动窗口最大滤波实现'最低、最左'的矩形装箱, 支持固定轮廓约束与自动选旋转。"

AI 回复 (摘要): 给出 packing.py 核心代码, 含积分图式空闲判定与向量化放置逻辑, 人工核验后采用。

交互记录 7 (代码辅助):

提示词 (Prompt): ” 请实现无重叠凸松弛下界: 对每模块按加权中位数做 x/y 坐标下降, 输出 HPWL 全局下界。”

AI 回复 (摘要): 给出 hpwl.py 中 convex_lb 实现, 人工核验收敛性与数值后采用。

交互记录 8 (代码辅助):

提示词 (Prompt): ” 请实现问题三的整数二分 + 可行性双证书: 启发式装箱给可行证书、CP-SAT 给不可行证书, 并加入增量收缩 warm start。”

AI 回复 (摘要): 给出 solve_q3.py 中二分主流程与可行性 oracle, 人工调试参数后采用。

四、采纳和人工修改情况

序号	AI 生成内容概述	是否采纳	人工修改情况说明
1	四层实施框架示意图初稿	部分采纳	人工补充文字标注、统一配色与箭头样式
2	问题二结果分析润色文本	部分采纳	人工逐句核对数据、调整措辞与结论表述

附录

附录 1：问题一求解核心代码（Skyline 装箱与四层框架）

```
# 本程序及代码是在 AI 工具辅助下完成的。
# AI 工具名称: DeepSeek, 版本/型号: DeepSeek-V4-Flash, 开发机构/公司: 深度求索 (DeepSeek),
# 版本发布日期: 2026-08-08。
"""核心几何算子: Skyline 装箱、字典序目标、合法性校验。

数据结构采用"稠密等高线数组 + 滑动窗口最大滤波":
* 轮廓线 contour[x] 表示 x 列当前的最高占用高度;
* 放置 w×h 矩形时, 滑动窗口最大滤波给出所有候选 x 处窗口内最大高度,
  取"最低、最左"位置放置 (即经典 Skyline/Bottom-Left 准则), 整体向量化。
"""

from __future__ import annotations

import numpy as np
from scipy.ndimage import maximum_filter1d

BIG = 10_000_000 # 字典序中面积项的放大系数, 使面积差严格支配长宽比差

def lex_cost(W: int, H: int) -> int:
    """字典序目标  $cost = WH \cdot BIG + \text{round}(1e6 \cdot |W/H - 1|)$ 。"""
    return W * H * BIG + int(round(abs(W / H - 1.0) * 1e6))

def lex_key(W: int, H: int) -> tuple[int, int]:
    """字典序比较键: (面积, 长宽比惩罚)。"""
    return W * H, round(abs(W / H - 1.0) * 1e6)

def pack_skyline(
    order: list[int],
    rot: list[int] | np.ndarray,
    widths: np.ndarray,
    heights: np.ndarray,
    bound_w: int,
    box: tuple[int, int | None] | None = None,
    auto_rotate: bool = False,
    ideal_x: np.ndarray | None = None,
):
    """按给定放置顺序与旋转, 用 Skyline 准则装箱。

    放置准则: 在"最低"可行位置中, 选取"该矩形下方残留空白面积最小"者 (填平凹凸),
    并列时取最左, 从而在经典 BL 基础上显著提升密度。
    若给定 ideal_x (凸松弛理想中心横坐标), 则在"最低"层内优先取离理想中心
```

最近的 x (PeF: 松弛放置 $\rightarrow$ 合法化), 使合法化后的布局贴近 HPWL 最优。

Args:

order: 模块序号放置顺序 (长度 n)。
rot: *per-module* 是否 90° 旋转 (0/1, 长度 n)。
widths/heights: 按模块序号索引的宽/高。
bound_w: 等高线数组宽度上界 (需 $\geq$ 任意可行轮廓宽)。
box: 可选 (W_0, H_0), 要求最终轮廓不超界; 超界则返回 *infeasible*。
auto_rotate: 若 *True*, 放置每个模块时贪心选择更优朝向并同步更新 *rot*。
ideal_x: 可选 *per-module* 理想横坐标 (中心), 用于 PeF 合法化。

Returns:

(xs, ys, rw, rh, W, H) 或 *None* (*box* 下不可行)。

"""

```
n = len(order)
Wmax = int(bound_w)
contour = np.zeros(Wmax + 1, dtype=np.int64)
xs = np.zeros(n, dtype=np.int64)
ys = np.zeros(n, dtype=np.int64)
rw = np.zeros(n, dtype=np.int64)
rh = np.zeros(n, dtype=np.int64)
rot = np.asarray(rot, dtype=np.int64)

for i, mid in enumerate(order):
    orientations = [(widths[mid], heights[mid]), (heights[mid], widths[mid])]
    if not auto_rotate:
        orientations = [orientations[int(rot[mid])]]
    best_place = None
    for wi, hi in orientations:
        if wi > Wmax:
            continue
        M = maximum_filter1d(contour, size=wi, origin=-(wi - 1) // 2,
                              mode="constant", cval=0)
        cand = M[: Wmax - wi + 1]
        if box is not None:
            W0, H0 = box
            ok_mask = np.arange(cand.size) <= W0 - wi
            if H0 is not None:
                ok_mask &= cand <= H0 - hi
            if not ok_mask.any():
                continue
            feasible = np.nonzero(ok_mask)[0]
            cand = cand[feasible]
        else:
            feasible = np.arange(cand.size)
        y = int(cand.min())
        y_ok = cand == y
        xs_cand = feasible[y_ok]
```

```

    if ideal_x is not None:
        # 最低层内取离理想中心最近的 x (PeF 合法化)
        target =int(ideal_x[mid]) -wi //2
        x =int(xs_cand[int(np.argmin(np.abs(xs_cand -target))))
    else:
        x =int(feasible[int(np.argmax(cand ==y))])
    if best_place is None or (y, x) <(best_place[0], best_place[1]):
        best_place =(y, x, wi, hi)
    if best_place is None:
        return None
    y, x, wi, hi =best_place
    rot[mid] =1 if (wi, hi) ==(heights[mid], widths[mid]) else 0
    xs[mid] =x
    ys[mid] =y
    rw[mid] =wi
    rh[mid] =hi
    contour[x :x +wi] =y +hi

W =int(np.max(xs +rw))
H =int(np.max(ys +rh))
return xs, ys, rw, rh, W, H

def verify(block_area: np.ndarray, xs: np.ndarray, ys: np.ndarray,
          rw: np.ndarray, rh: np.ndarray, W: int, H: int) ->tuple[bool, int]:
    """合法性与重叠检查 ( $O(n^2)$ )。返回 (合法, 最大重叠面积)"""
    n =len(xs)
    if n ==0:
        return True, 0
    if xs.min() <0 or ys.min() <0 or int((xs +rw).max()) >W or int((ys +rh).max())
        >H:
        return False, -1
    max_overlap =0
    for i in range(n):
        for j in range(i +1, n):
            ox =min(xs[i] +rw[i], xs[j] +rw[j]) -max(xs[i], xs[j])
            oy =min(ys[i] +rh[i], ys[j] +rh[j]) -max(ys[i], ys[j])
            if ox >0 and oy >0:
                max_overlap =max(max_overlap, ox *oy)
    return max_overlap ==0, max_overlap

def total_area(block_area: np.ndarray) ->int:
    return int(block_area.sum())

```

本程序及代码是在 AI 工具辅助下完成的。
 # AI 工具名称: DeepSeek, 版本/型号: DeepSeek-V4-Flash, 开发机构/公司: 深度求索(DeepSeek),
 版本发布日期: 2026-08-08。

"""问题一主求解入口：四层框架落地。

L1 多源初解池 → L2 并行多目标元启发 (Skyline+ILS 主 / B*-树+SA 对照)
→ L3 双向反馈定界 (密度 gap / CP-SAT 子集证书) → L4 配置适配。

输出：

- * 三组芯片的轮廓面积、长宽比、密度与摆放可视化；
- * "多起点 vs 单起点"收敛对比表与曲线；
- * "密度 gap"表 (上界证书 / 下界)。

"""

```
from __future__ import annotations
```

```
import argparse
```

```
import json
```

```
import os
```

```
import sys
```

```
import time
```

```
import numpy as np
```

```
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
```

```
from config import SCENARIOS # noqa: E402
```

```
from data_io import load_dataset, dataset_stats # noqa: E402
```

```
from layer1_pool import generate_initial_pool, diversity_filter # noqa: E402
```

```
from layer2_search import ( # noqa: E402
    width_bounds, width_grid, ils_trajectory, final_width_sweep,
    strong_perturb, run_parallel, _worker_area, _worker_bstar,
)
```

```
from layer3_bound import ( # noqa: E402
    lower_bounds, gap_report, check_feasible, cp_sat_subset_feasible,
)
```

```
from bstar import BStarTree # noqa: E402
```

```
from visualize import plot_layout, plot_convergence # noqa: E402
```

```
def solve_dataset(ds, cfg):
```

```
    widths = np.asarray(ds.widths, dtype=np.int64)
```

```
    heights = np.asarray(ds.heights, dtype=np.int64)
```

```
    n = ds.n
```

```
    A = ds.total_area
```

```
    w_lb, w_ub = width_bounds(widths, heights)
```

```
    wgrid = width_grid(widths, heights, n=max(5, min(8, cfg.kp)))
```

```
    W0 = int(round(np.sqrt(A)))
```

```
    t0 = time.time()
```

```
    # ---- L1 多源初解生成池 + 多样性粗筛 ----
```

```
    rng = np.random.default_rng(cfg.seed)
```

```
    pool = generate_initial_pool(ds, rng, prob_rot=0.3, strip_w=W0)
```



```

pool =diversity_filter(pool, cfg.kp)

# ---- L2 主算法: Skyline+ILS 多起点并行 (固定条带宽, 覆盖宽度网格) ----
tasks = [
    (_worker_area, dict(widths=widths, heights=heights, order=sol["order"],
                        rot=sol["rot"].tolist(),
                        W=wgrid[i % len(wgrid)], budget_s=cfg.t_ils,
                        seed=cfg.seed + i *101 +7, phase="area"))
    for i, sol in enumerate(pool)
]
results = [r for r in run_parallel(tasks, cfg.nproc) if r is not None]

# ---- L2 精英池 + 扰动重启 (在精英解宽度邻域内) ----
elites =sorted(results, key=lambda r: (r["area"], r["asp"]))[:3]
rng2 =np.random.default_rng(cfg.seed +999)
restart_tasks = []
for j, el in enumerate(elites):
    o2, r2 =strong_perturb(el["order"], np.asarray(el["rot"]), n, rng2)
    Wr =max(w_lb, min(w_ub, el["W"] +(j -1) *3))
    restart_tasks.append(
        (_worker_area, dict(widths=widths, heights=heights, order=o2,
                            rot=r2.tolist(), W=Wr, budget_s=cfg.t_restart,
                            seed=cfg.seed +5000 +j *17, phase="area"))
    )
results += [r for r in run_parallel(restart_tasks, cfg.nproc) if r is not None]

# ---- L2 长宽比精调 (字典序第二级: 面积不增, 压 |W/H-1|) ----
s_area =min(r["area"] for r in results)
aspect_tasks = []
for j, el in enumerate(elites):
    Wasp =max(w_lb, min(w_ub, el["W"] +(j -1) *2))
    aspect_tasks.append(
        (_worker_area, dict(widths=widths, heights=heights, order=el["order"],
                            rot=el["rot"], W=Wasp, budget_s=cfg.t_aspect,
                            seed=cfg.seed +10000 +j *13, phase="aspect",
                            s_area=s_area))
    )
results += [r for r in run_parallel(aspect_tasks, cfg.nproc) if r is not None]

# ---- L2 对照算法: B*-树 + 模拟退火 ----
bound_b =int(max(widths.sum(), heights.sum())) +10
bstar_tasks = []
for j, el in enumerate(elites[:2]):
    tree =BStarTree.from_shelf(widths, heights, el["order"],
                               np.asarray(el["rot"]), int(el["W"]))
    bstar_tasks.append(
        (_worker_bstar, dict(widths=widths, heights=heights, rot=tree.rot.tolist
                               (),

```

```

        root=int(tree.root), parent=tree.parent.tolist(),
        left=tree.left.tolist(), right=tree.right.tolist(),
        bound_w=bound_b, budget=cfg.t_bstar,
        seed=cfg.seed +7000 +j *19))
    )
bstar_results =[r for r in run_parallel(bstar_tasks, cfg.nproc) if r is not
                None]

# ---- 全局条带宽全扫：落实字典序（面积最小 → 长宽比最接近1）----
top =sorted(results, key=lambda r: (r["area"], r["asp"]))[:5]
finals =[]
for r in top:
    f =final_width_sweep(r["order"], r["rot"], widths, heights, w_lb, w_ub,
        phase="area")
    if f is not None:
        finals.append(f)
if bstar_results:
    bb =min(bstar_results, key=lambda r: (r["area"], r["asp"]))
    finals.append(bb)
final =min(finals, key=lambda r: (r["area"], r["asp"]))

# ---- L3 校验与定界 ----
ok, _ =check_feasible(ds, final["xs"], final["ys"], final["rw"], final["rh"],
    final["W"], final["H"])
lb =lower_bounds(ds)
gap =gap_report(ds, final["W"], final["H"])
cp =cp_sat_subset_feasible(ds, final["W"], final["H"], cfg.cp_subset,
    cfg.cp_timeout)
runtime =time.time() -t0

bstar_best =min(bstar_results, key=lambda r: (r["area"], r["asp"])) \
    if bstar_results else None
return dict(
    dataset=ds.name, final=final, feasible=ok, bounds=lb, gap=gap, cp=cp,
    runtime=runtime, traces=[r["trace"] for r in results],
    results_area=[(r["area"], r["asp"]) for r in results],
    bstar=(None if bstar_best is None else
        (bstar_best["W"], bstar_best["H"], bstar_best["area"],
        bstar_best["asp"])),
    pool=[(s["key"], s["W"], s["H"], s["area"], s["cost"]) for s in pool],
)

def _fmt_area_table(results):
    rows =[]
    for r in results:
        A =r["bounds"]["A"]
        rows.append((r["dataset"], r["final"]["W"], r["final"]["H"],

```

```

        r["final"]["area"], A, r["gap"]["density"],
        r["gap"]["gap_pct"], r["final"]["asp"], r["feasible"])))
    return rows

def _save_placement(ds, final, out_dir):
    """保存模块摆放结果文件（名称 x y w h 是否旋转）。"""
    lines = [f"# {ds.name} Q1 placement W={final['W']} H={final['H']}"]
    for i in range(ds.n):
        lines.append(f"{ds.names[i]} {final['xs'][i]} {final['ys'][i]} "
                     f"{final['rw'][i]} {final['rh'][i]} {int(final['rot'][i])}")
    os.makedirs(out_dir, exist_ok=True)
    with open(os.path.join(out_dir, f"q1_{ds.name}_placement.txt"), "w",
              encoding="utf-8") as fh:
        fh.write("\n".join(lines))

def main():
    here = os.path.dirname(os.path.abspath(__file__))
    parser = argparse.ArgumentParser(description="问题一：最小面积/长宽比 Packing")
    parser.add_argument("--data-dir", default=os.path.normpath(os.path.join(here, "..", "附件")))
    parser.add_argument("--out-dir", default=os.path.normpath(os.path.join(here, "..", "result")))
    parser.add_argument("--datasets", default="n100,n200,n300")
    parser.add_argument("--nproc", type=int, default=6)
    parser.add_argument("--scale", type=float, default=1.0,
                        help="时间预算缩放系数（快速测试用）")
    parser.add_argument("--cp", action="store_true", help="启用 CP-SAT 子集证书")
    args = parser.parse_args()

    cfg = SCENARIOS["Q1"]
    cfg.nproc = args.nproc
    cfg.out_dir = args.out_dir
    cfg.t_ils *= args.scale
    cfg.t_bstar *= args.scale
    cfg.t_aspect *= args.scale
    cfg.t_restart *= args.scale
    if not args.cp:
        cfg.cp_timeout = 0
    cfg.kp = max(2, int(round(cfg.kp * args.scale)))

    datasets = [s.strip() for s in args.datasets.split(",") if s.strip()]
    results = []
    for name in datasets:
        ds = load_dataset(args.data_dir, name)
        print(f"\n==== 数据集 {name} {dataset_stats(ds)} =====", flush=True)
        r = solve_dataset(ds, cfg)

```

```

results.append(r)
f =r["final"]
print(f" 轮廓: {f['W']} × {f['H']} 面积: {f['area']} "
      f"密度: {r['gap']['density']*100:.2f}%"
      f"长宽比: {f['asp']:.3f} 可行: {r['feasible']}", flush=True)

# ---- 结果表 ----
rows =_fmt_area_table(results)
print("\n" + "=" *82)
print("表1 三组芯片问题一求解结果")
print("=" *82)
print(f"{'数据集':<8}{'轮廓W':>7}{'轮廓H':>7}{'面积S':>9}{'总面积A':>10}"
      f"{'密度p':>9}{'gap(1-p)':>10}{'长宽比':>9}{'可行':>6}")
for row in rows:
    ds, W, H, S, A, rho, gpct, asp, ok =row
    print(f"{'ds':<8}{'W':>7}{'H':>7}{'S':>9}{'A':>10}{'rho*100':>8.2f}{'gpct':>9.2f}%"
          f"{'asp':>9.3f}{'是' if ok else '否':>6}")

lb_rows =[r["dataset"], r["bounds"]["A"], r["bounds"]["S_lb"],
          r["final"]["area"], r["final"]["area"] /r["bounds"]["S_lb"])
    for r in results]
print("\n" + "=" *82)
print("表2 密度 gap 表（下界与可行证书）")
print("=" *82)
print(f"{'数据集':<8}{'面积下界S_lb':>14}{'上界(证书)S_ub':>16}{'gap':>10}")
for name, A, S_lb, S_ub, _ in lb_rows:
    print(f"{'name':<8}{'S_lb':>14}{'S_ub':>16}{'(S_ub/S_lb-1)*100':>9.2f}%"

# ---- 多起点 vs 单起点收敛表 ----
print("\n" + "=" *100)
print("表3 多起点与单起点收敛对比（各时间点已得最优面积，越小越好）")
print("=" *100)
hdr =f"{'数据集':<8}{'起点数':>6}{'t=25%':>12}{'t=50%':>12}{'t=75%':>12}{'t=100%":>12}"
print(hdr +"（多起点累积最优）" + "单起点(最优轨迹)")
for r in results:
    traces =r["traces"]
    tmax =max((tr[-1][0] for tr in traces), default=1.0)
    fracs =(0.25, 0.5, 0.75, 1.0)
    multi =[]
    for frac in fracs:
        t =frac *tmax
        best =min((a for tr in traces for tt, a, _ in tr if tt <=t),
                  default=float("inf"))
        multi.append(best)
    single_row =[]
    if traces:
        sorted_tr =sorted(traces, key=lambda tr: tr[0][1])

```

```

tr =sorted_tr[len(sorted_tr) //2] # 中位初解 (典型单跑)
for frac in fracs:
    vals =[a for tt, a, _ in tr if tt <=frac *tmax]
    single_row.append(vals[-1] if vals else tr[0][1])
def fmt(v):
    return "—" if v ==float("inf") else f"{v:>12,}"
print(f"{r['dataset']:<8}{len(traces):>6}"
      f"{fmt(multi[0])}{fmt(multi[1])}{fmt(multi[2])}{fmt(multi[3])}"
      f" | {fmt(single_row[0])}{fmt(single_row[1])}{fmt(single_row[2])}{fmt(
          single_row[3])}")

# ---- 主算法 vs 对照算法 (B*-树+SA) ----
print("\n" + "=" *100)
print("表4 主算法 Skyline+ILS 与 对照算法 B*-树+SA 结果对比")
print("=" *100)
print(f"{'数据集':<8}{ 'ILS 轮廓':>12}{ 'ILS 面积':>12}{ 'ILS 密度':>10}"
      f"{'B*树 轮廓':>12}{ 'B*树 面积':>12}{ 'B*树 密度':>10}")
for r in results:
    f =r["final"]
    bb =r["bstar"]
    bstr =f"{bb[0]}×{bb[1]}" if bb else "—"
    barea =f"{bb[2]:,}" if bb else "—"
    bd =f"{r['bounds']['A'] / bb[2] * 100:.2f}%" if bb else "—"
    print(f"{r['dataset']:<8}{f'{f['W']}]×{f['H']}]':>12}{f['area']:>12,}"
          f"{r['gap']['density'] * 100:>9.2f}%{bstr:>12}{barea:>12}{bd:>10}")

# ---- CP-SAT 子集证书 ----
if args.cp:
    print("\n" + "=" *100)
    print("表5 CP-SAT 小规模子集可行性证书 (L3 定界验证)")
    print("=" *100)
    for r in results:
        status, cnt, el =r["cp"]
        print(f"{r['dataset']:<8} 子集规模(面积最大块数): {cnt:>3} 状态: {status:<10}"
              "
              f"耗时: {el:.1f}s")

# ---- 可视化 ----
for r in results:
    ds_name =r["dataset"]
    f =r["final"]
    plot_layout(
        load_dataset(args.data_dir, ds_name), f["xs"], f["ys"], f["rw"], f["rh"],
        f["W"], f["H"],
        os.path.join(args.out_dir, f"q1_{ds_name}_layout.png"),
        title=f"问题一 {ds_name} 模块摆放",
        density=r["gap"]["density"], aspect=f["asp"])
    plot_convergence(r["traces"],

```

```

os.path.join(args.out_dir, f"q1_{ds_name}_convergence.png"))

# ---- 保存结果文件 ----
summary = []
for r in results:
    f = r["final"]
    ds = load_dataset(args.data_dir, r["dataset"])
    _save_placement(ds, f, args.out_dir)
    summary.append(dict(
        dataset=r["dataset"], n=ds.n, A=r["bounds"]["A"],
        S_lb=r["bounds"]["S_lb"], W=f["W"], H=f["H"], area=f["area"],
        density=round(r["gap"]["density"], 6),
        gap=round(r["gap"]["gap_pct"], 4), aspect=round(f["asp"], 6),
        feasible=r["feasible"], runtime=round(r["runtime"], 1),
        bstar=(dict(W=r["bstar"][0], H=r["bstar"][1], area=r["bstar"][2])
                if r["bstar"] else None),
    ))
os.makedirs(args.out_dir, exist_ok=True)
with open(os.path.join(args.out_dir, "q1_summary.json"), "w",
          encoding="utf-8") as fh:
    json.dump(summary, fh, ensure_ascii=False, indent=2)
print(f"\n结果与可视化已输出至: {os.path.abspath(args.out_dir)}")

if __name__ == "__main__":
    main()

```

附录

附录 1：问题二求解核心代码（线网索引、凸松弛下界与 Skyline-ILS/SA 主算法）

```
# 本程序及代码是在 AI 工具辅助下完成的。
# AI 工具名称: DeepSeek, 版本/型号: DeepSeek-V4-Flash, 开发机构/公司: 深度求索(DeepSeek),
# 版本发布日期: 2026-08-08。
"""问题二共享核心: 线网索引 + HPWL 评估 (全量/增量) + 凸松弛下界。

模块划分:
* NetIndex —— 线网→引脚、模块→线网两级索引; 全量 HPWL 与 "仅重算被扰
    模块所涉线网"的增量评估  $O(deg(i) \cdot |\bar{d}|)$ ;
* overflow —— 固定轮廓越界量 (L2 罚函数用);
* convex_lb —— 无重叠凸松弛下界 (加权中位数坐标下降, L3 定界证书)。
"""

from __future__ import annotations

import numpy as np

class NetIndex:
    """维护线网与模块/终端的关联关系。

    引脚命名约定 (与附件一致): 模块引脚名以 'b' 开头, 终端引脚名以 'p' 开头。
    模块引脚位于模块几何中心 (随摆放位置变化); 终端引脚坐标固定 (.pl 给出)。
    """

    def __init__(self, names: list[str], nets: list[list[str]],
                  terminal_pos: dict[str, tuple[int, int]]):
        self.names = list(names)
        self.n = len(self.names)
        self.name2idx = {nm: i for i, nm in enumerate(self.names)}
        self.terminal_pos = terminal_pos
        self.nets = nets
        # net_pins[k] = [(kind, idx), ...] kind in {'m'|'t'}
        # 'm' -> 模块下标; 't' -> 终端固定坐标 (x, y)
        self.net_pins: list[list[tuple[str, object]]] = []
        # mod_nets[i] = 与模块 i 相连的线网下标
        self.mod_nets: list[list[int]] = [[] for _ in range(self.n)]
        for k, net in enumerate(nets):
            pins = []
            for pid in net:
                if pid.startswith("b"):
                    pins.append(("m", self.name2idx.get(pid)))
                else:
                    pins.append(("t", self.terminal_pos.get(pid)))
            self.net_pins.append(pins)
```

```

        for kind, ref in pins:
            if kind == "m" and ref is not None:
                self.mod_nets[ref].append(k)

# ----- #
# 引脚坐标收集
# ----- #
def pin_coords(self, k: int, xs, ys, rw, rh):
    """返回线网 k 的全部引脚 (x, y) 坐标列表。"""
    pts = []
    for kind, ref in self.net_pins[k]:
        if kind == "m":
            if ref is None:
                continue
            pts.append((xs[ref] + rw[ref] / 2.0, ys[ref] + rh[ref] / 2.0))
        else:
            if ref is None:
                continue
            pts.append((float(ref[0]), float(ref[1])))
    return pts

# ----- #
# 全量 / 单线网 HPWL
# ----- #
def net_hpwl(self, k: int, xs, ys, rw, rh) -> float:
    xs_p = []
    ys_p = []
    for kind, ref in self.net_pins[k]:
        if kind == "m":
            if ref is None:
                continue
            xs_p.append(xs[ref] + rw[ref] / 2.0)
            ys_p.append(ys[ref] + rh[ref] / 2.0)
        else:
            if ref is None:
                continue
            xs_p.append(float(ref[0]))
            ys_p.append(float(ref[1]))
    if not xs_p:
        return 0.0
    return (max(xs_p) - min(xs_p)) + (max(ys_p) - min(ys_p))

def total_hpwl(self, xs, ys, rw, rh) -> float:
    return sum(self.net_hpwl(k, xs, ys, rw, rh) for k in range(len(self.nets)))

# ----- #
# 增量 HPWL (核心加速: 只重算被扰动模块所涉线网)
# ----- #

```



```

def delta_hpwl(self, touched_nets, xs, ys, rw, rh,
               xs_old, ys_old, rw_old, rh_old) ->float:
    """给定被扰动线网集合，计算新旧布局 HPWL 之差。

    touched_nets 由调用方根据"发生变化的模块"推导（见 bstar_q2）。
    """
    delta =0.0
    for k in touched_nets:
        delta +=(self.net_hpwl(k, xs, ys, rw, rh)
                - self.net_hpwl(k, xs_old, ys_old, rw_old, rh_old))
    return delta

# ----- #
# 启发式辅助：线网度 / 终端相连线网
# ----- #
def module_netdeg(self) ->np.ndarray:
    return np.asarray([len(v) for v in self.mod_nets], dtype=np.int64)

def module_termdeg(self) ->np.ndarray:
    """每模块相连线网中"含终端"的线网数（Terminal 引力强度近似）。"""
    term =np.zeros(self.n, dtype=np.int64)
    for i in range(self.n):
        for k in self.mod_nets[i]:
            if any(kind =="t" for kind, _ in self.net_pins[k]):
                term[i] +=1
    return term

# ----- #
# 固定轮廓越界量（L2 罚函数）
# ----- #
def overflow(xs, ys, rw, rh, Lhat: int) ->int:
    """越界量 =  $\sum_i [\max(0, x+w-Lhat) + \max(0, y+h-Lhat)]$ 。"""
    xs =np.asarray(xs, dtype=np.int64)
    ys =np.asarray(ys, dtype=np.int64)
    rw =np.asarray(rw, dtype=np.int64)
    rh =np.asarray(rh, dtype=np.int64)
    return int(np.maximum(0, xs +rw -Lhat).sum()
               + np.maximum(0, ys +rh -Lhat).sum())

def overflow_of_module(i, xs, ys, rw, rh, Lhat: int) ->int:
    return (max(0, int(xs[i]) +int(rw[i]) -Lhat)
            + max(0, int(ys[i]) +int(rh[i]) -Lhat))

def hpwl_of_placement(net: NetIndex, xs, ys, rw, rh) ->float:
    return net.total_hpwl(xs, ys, rw, rh)

```

```

# ----- #
# L3: 凸松弛下界（无重叠松弛 + 加权中位数坐标下降）
# ----- #
def convex_lb(ds, Lhat: int, max_iter: int =300, tol: float =1e-6,
              boundary: bool =False) ->float:
    """无重叠凸松弛问题  $\min \square$  HPWLk（只保留固定轮廓约束）的全局下界。

    理论依据（问题2建模性质3）：对单模块横坐标  $x_i$ ，其贡献为若干 hinge 函数
     $\max(0, x-h, l-x)$  之和，最优值在断点加权中位数处取得；坐标下降收敛到全局最优。
    boundary=True 时进一步约束模块中心到轮廓边界的最小距离（模块须完整位于
    轮廓内），得到更紧的 LB1 下界；boundary=False 为仅去掉非重叠约束的 LB0。
    返回松弛问题最优值（原问题总 HPWL 的全局下界）。
    """
    return _convex_lb_impl(ds, Lhat, max_iter, tol, boundary)[0]

def convex_lb_with_centers(ds, Lhat: int, max_iter: int =300,
                          tol: float =1e-6, boundary: bool =False):
    """返回 (lb_value, centers)。centers[i] = 模块 i 的松弛理想中心 (x, y)。

    该理想中心被 L1 用于构建"终端感知/松弛中心排序"初解（PeF 范式：
    松弛放置 → 合法化），使合法化后的布局天然贴近 HPWL 最优。
    """
    return _convex_lb_impl(ds, Lhat, max_iter, tol, boundary)

def _convex_lb_impl(ds, Lhat, max_iter, tol, boundary=False):
    names =list(ds.names)
    name2idx ={nm: i for i, nm in enumerate(names)}
    n =ds.n
    L =float(Lhat)

    # 模块初始中心放在轮廓中心
    cx =np.full(n, L /2.0)
    cy =np.full(n, L /2.0)

    # 每个模块：相连线网 -> 每线网的其它引脚坐标
    mod_nets: dict[int, list[int]] ={i: [] for i in range(n)}
    net_pins: list[list[tuple[str, object]]] =[]
    for net in ds.nets:
        pins =[]
        for pid in net:
            if pid.startswith("b"):
                j =name2idx.get(pid)
                if j is None:
                    continue

```

```

        pins.append(("m", j))
    else:
        pos = ds.terminal_pos.get(pid)
        if pos is None:
            continue
        pins.append(("t", pos))
    k = len(net_pins)
    net_pins.append(pins)
    for kind, ref in pins:
        if kind == "m":
            mod_nets[ref].append(k)

def _others_x(k, exclude):
    xs_p = []
    for kind, ref in net_pins[k]:
        if kind == "m":
            if ref == exclude:
                continue
            xs_p.append(cx[ref])
        else:
            xs_p.append(float(ref[0]))
    return xs_p

def _others_y(k, exclude):
    ys_p = []
    for kind, ref in net_pins[k]:
        if kind == "m":
            if ref == exclude:
                continue
            ys_p.append(cy[ref])
        else:
            ys_p.append(float(ref[1]))
    return ys_p

def total():
    tot = 0.0
    for pins in net_pins:
        xs_p = [cx[r] if t == "m" else float(r[0]) for t, r in pins]
        ys_p = [cy[r] if t == "m" else float(r[1]) for t, r in pins]
        if xs_p:
            tot += (max(xs_p) - min(xs_p)) + (max(ys_p) - min(ys_p))
    return tot

# 加权中位数坐标下降 (Gauss-Seidel 扫描, x/y 交替)
for _ in range(max_iter):
    prev = total()
    for axis in (0, 1):
        for i in range(n):

```

```

bps = []
for k in mod_nets[i]:
    others = _others_x(k, i) if axis == 0 else _others_y(k, i)
    if others:
        bps.append(min(others))
        bps.append(max(others))
if not bps:
    c = L / 2.0
else:
    bps.sort()
    c = bps[len(bps) // 2]
if boundary:
    # LB1: 模块中心须满足"模块完整位于轮廓内"的最小边距
    m = min(ds.widths[i], ds.heights[i]) / 2.0
    c = min(max(c, m), L - m)
else:
    c = min(max(c, 0.0), L)
if axis == 0:
    cx[i] = c
else:
    cy[i] = c
if abs(total() - prev) < tol:
    break
centers = [(cx[i], cy[i]) for i in range(n)]
return total(), centers

```

本程序及代码是在 AI 工具辅助下完成的。

AI 工具名称: DeepSeek, 版本/型号: DeepSeek-V4-Flash, 开发机构/公司: 深度求索 (DeepSeek),
版本发布日期: 2026-08-08。

"""第2层 (L2) 问题二主算法: Skyline 装箱 + 迭代局部搜索/模拟退火 (ILS/SA)。

关键设计 (针对固定轮廓 + HPWL 目标):

- * Skyline 装箱 $box=(L_{hat}, L_{hat})$ 约束内建 —— 每次评估都是可行解, 不浪费迭代去"修复越界", 全部预算用于压低总 HPWL;
- * 解表示以放置顺序为主; 模块朝向由 Skyline 解码器 ($auto_rotate=True$) 依据"最低、最左"局部准则自适应确定, 扰动算子 (轻/中/重三档) 重排放置顺序以驱动搜索;
- * Metropolis 接受准则 (SA 降温) 跳出局部最优;
- * HPWL 评估统一走 NetIndex 精确半坐标口径 (与 L1/L3 一致)。

本算法在固定轮廓可行域内直接搜索, 三组数据实测 HPWL 优于 B^* -树+SA。

"""

```
from __future__ import annotations
```

```
import time
```

```
import numpy as np
```

```
from packing import pack_skyline
```

```

from hpwl import NetIndex

def pack_and_eval(order, rot, widths, heights, net: NetIndex, Lhat):
    """在固定轮廓内 Skyline 装箱 (auto_rotate=True) 并返回 (xs,ys,rw,rh,hpwl)。


装箱失败 (轮廓内放不下) 返回 None。


    """
    res = pack_skyline(order, rot, widths, heights, Lhat,
                       box=(Lhat, Lhat), auto_rotate=True)
    if res is None:
        return None
    xs, ys, rw, rh, W, H = res
    hpwl = net.total_hpwl(xs, ys, rw, rh)
    return xs, ys, rw, rh, hpwl

def perturb_order_rot(order, rot, n, rng, strength="medium"):
    """扰动放置顺序与旋转 (light/medium/heavy 三档强度) 。
    order2 = list(order)
    rot2 = np.asarray(rot, dtype=np.int64).copy()

    if strength == "light":
        k = max(2, n // 20)
        n_flip = max(1, n // 15)
    elif strength == "heavy":
        k = max(3, n // 6)
        n_flip = max(2, n // 5)
    else:
        k = max(2, n // 10)
        n_flip = max(1, n // 8)

    idx = sorted(rng.choice(n, min(k, n), replace=False), reverse=True)
    vals = [order2[i] for i in idx]
    for v in vals:
        order2.remove(v)
    rng.shuffle(vals)
    posns = sorted(rng.integers(0, len(order2) + 1, size=len(vals)))
    for v, p in zip(vals, posns):
        order2.insert(p, v)

    flip_idx = rng.choice(n, min(n_flip, n), replace=False)
    rot2[flip_idx] ^= 1

    if strength in ("medium", "heavy") and rng.random() < 0.5:
        a = int(rng.integers(0, n))
        b = int(rng.integers(0, n))
        lo, hi = min(a, b), max(a, b)

```

```

        order2[lo:hi +1] =order2[lo:hi +1][::-1]

    return order2, rot2

def ils_hpwl_trajectory(widths, heights, order, rot, net, Lhat, budget_s,
                        seed=0, trace_interval=1.0):
    """单条 Skyline+ILS/SA 轨迹: 从 (order, rot) 出发优化 HPWL。

    返回 dict(xs, ys, rw, rh, hpwl, order, rot, iterations, trace, feasible)。
    trace = [(t, best_hpwl, overflow=0), ...]。
    """
    rng =np.random.default_rng(seed)
    n =len(widths)
    widths =np.asarray(widths, dtype=np.int64)
    heights =np.asarray(heights, dtype=np.int64)

    init =pack_and_eval(order, rot, widths, heights, net, Lhat)
    if init is None:
        # 初序在 (Lhat,Lhat) 内放不下: 尝试找可行初解, 仍失败则放弃该轨迹
        from packing import pack_skyline
        found =None
        for _ in range(40):
            o2, r2 =perturb_order_rot(order, rot, n, rng, "heavy")
            res =pack_skyline(o2, r2, widths, heights, Lhat,
                             box=(Lhat, Lhat), auto_rotate=True)

            if res is not None:
                xs0, ys0, rw0, rh0, W0, H0 =res
                found =(o2, r2, xs0, ys0, rw0, rh0,
                        net.total_hpwl(xs0, ys0, rw0, rh0))

                break
        if found is None:
            return None

        order, rot =list(found[0]), np.asarray(found[1], dtype=np.int64).copy()
        xs, ys, rw, rh, hpwl =found[2], found[3], found[4], found[5], found[6]
    else:
        xs, ys, rw, rh, hpwl =init
        order =list(order)
        rot =np.asarray(rot, dtype=np.int64).copy()

    best =dict(xs=xs, ys=ys, rw=rw, rh=rh, hpwl=hpwl,
              order=order, rot=rot.copy())

    # 自适应初温: 估计典型 HPWL 变化量
    deltas =[]
    for _ in range(30):
        o2, r2 =perturb_order_rot(order, rot, n, rng, "medium")
        r2_res =pack_and_eval(o2, r2, widths, heights, net, Lhat)

```

```

    if r2_res is not None:
        deltas.append(abs(r2_res[-1] -hpwl))
T0 =float(np.mean(deltas)) *2.0 +1.0 if deltas else float(hpwl) *0.01 +100.0

T =T0
T_min =1e-2
alpha =0.96
it =0
stagnant =0
max_stagnant =400

trace =[(0.0, best["hpwl"], 0)]
last_trace =0.0
start =time.time()

while time.time() -start <budget_s and T >T_min:
    inner =max(50, n *3)
    improved_inner =False
    for _ in range(inner):
        strength =("heavy" if rng.random() <0.08 else
                    "light" if rng.random() <0.3 else "medium")
        o2, r2 =perturb_order_rot(order, rot, n, rng, strength)
        r2_res =pack_and_eval(o2, r2, widths, heights, net, Lhat)
        if r2_res is None:
            continue
        hpwl2 =r2_res[-1]
        delta =hpwl2 -hpwl
        it +=1
        if delta <=0 or rng.random() <np.exp(-delta /T):
            order, rot =o2, r2
            hpwl =hpwl2
            if hpwl2 <best["hpwl"]:
                best =dict(xs=r2_res[0], ys=r2_res[1], rw=r2_res[2],
                           rh=r2_res[3], hpwl=hpwl2,
                           order=list(order), rot=rot.copy())
                improved_inner =True
    T *=alpha
    if improved_inner:
        stagnant =0
    else:
        stagnant +=1
    if stagnant >=max_stagnant:
        T =T0 *0.4
        stagnant =0
    for _ in range(max(5, n //8)):
        o2, r2 =perturb_order_rot(order, rot, n, rng, "heavy")
        r2_res =pack_and_eval(o2, r2, widths, heights, net, Lhat)
        if r2_res is not None:

```

```

        order, rot = o2, r2
        hpwl = r2_res[-1]
    now = time.time() - start
    if now - last_trace >= trace_interval:
        last_trace = now
        trace.append((now, best["hpwl"], 0))
    trace.append((time.time() - start, best["hpwl"], 0))

    best["iterations"] = it
    best["trace"] = trace
    best["feasible"] = True
    return best

```


附 录

附录 1：问题三求解核心代码（整数二分与可行性双证书）

```
# 本程序及代码是在 AI 工具辅助下完成的。
# AI 工具名称: DeepSeek, 版本/型号: DeepSeek-V4-Flash, 开发机构/公司: 深度求索(DeepSeek),
# 版本发布日期: 2026-08-08。
"""问题三主求解入口：最小死区比例（双证书整数二分 + 可行性夹逼）。

沿用问题一/二四层框架（L4 配置 Q3: outline=search, objective=min_L, nets=False）:
    L1 紧凑装箱初解（复用 Q1 机制）→ L2 整数二分 + 可行性证书
    （Skyline-ILS 可行 • CP-SAT 子集不可行）→ L3 双向反馈夹逼 → 第二阶
    段在 L* 下更新 HPWL（复用 Q2 流水线）。

输出：
    * 三组芯片最小可行边长 L*、最小死区比例 d*、夹逼区间与 gap;
    * L* 下总 HPWL（与 d=0.15 对比，量化死区压缩代价）；
    * 模块摆放可视化（Terminal 标注 + L*×L* 轮廓框）。
"""
from __future__ import annotations

import argparse
import json
import os
import sys
import time
import numpy as np
from concurrent.futures import ProcessPoolExecutor

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from config import SCENARIOS # noqa: E402
from data_io import load_dataset, dataset_stats # noqa: E402
from hpwl import NetIndex, overflow # noqa: E402
from packing import pack_skyline # noqa: E402
from layer1_pool_q2 import ( # noqa: E402
    generate_initial_pool_q2, diversity_filter_q2,
)
from skyline_ils_q2 import ils_hpwl_trajectory # noqa: E402
from layer3_bound_q3 import ( # noqa: E402
    lower_bound_L, upper_bound_L, d_of_L, density,
    gap_report_q3, cp_sat_subset_infeasible,
)
from layer3_bound_q2 import lower_bound_hpwl, check_feasible_q2, gap_report_q2 # noqa: E402
from solve_q2 import solve_dataset as solve_q2_dataset # noqa: E402
from visualize import plot_layout_q2, plot_convergence_q2 # noqa: E402
```

```

# ----- #
# 可行性 oracle (L1 多源 + L2 Skyline-ILS)
# ----- #
def _feas_orders(widths, heights, rng):
    """面向紧凑装箱的排序规则池 (10 种规则 + 随机)。"""
    n = len(widths)
    area = widths * heights
    longside = np.maximum(widths, heights)
    perimeter = widths + heights
    flexibility = np.abs(widths - heights)
    orders = {
        "area_desc": np.argsort(-area, kind="stable").tolist(),
        "long_desc": np.argsort(-longside, kind="stable").tolist(),
        "perim_desc": np.argsort(-perimeter, kind="stable").tolist(),
        "lff_desc": np.argsort(-flexibility, kind="stable").tolist(),
        "area_asc": np.argsort(area, kind="stable").tolist(),
        "long_asc": np.argsort(longside, kind="stable").tolist(),
        "width_desc": np.argsort(-widths, kind="stable").tolist(),
        "height_desc": np.argsort(-heights, kind="stable").tolist(),
        "maxside_asc": np.argsort(longside, kind="stable").tolist(),
        "min_side_desc": np.argsort(-np.minimum(widths, heights), kind="stable").
            tolist(),
    }
    for j in range(12):
        orders[f"rand_{j}"] = rng.permutation(n).tolist()
    return orders

def pack_feasible(widths, heights, order, rot, L, ideal_x=None):
    """在 (L,L) 内 Skyline 装箱, 返回 (xs,ys,rw,rh,ovf) 或 None。"""
    res = pack_skyline(order, rot, widths, heights, L,
        box=(L, L), auto_rotate=True, ideal_x=ideal_x)
    if res is None:
        return None
    xs, ys, rw, rh, W, H = res
    ovf = overflow(xs, ys, rw, rh, L)
    return xs, ys, rw, rh, ovf

def feasible_oracle(ds, widths, heights, net, L, cfg, seed,
    ideal_x=None, budget_ils=None, warm_seed=None):
    """三态可行性 oracle: 返回 (status, layout)。

    status ∈ {'feasible', 'infeasible', 'unknown'}。
    layout = dict(xs,ys,rw,rh,hpwl,order,rot,overflow) 或 None。
    warm_seed: 上一可行 L 的解 dict(order, rot), 原样作为本 L 的种子

```

```

        (增量收缩 warm start, 避免每层 L 从头搜索)。
"""
rng = np.random.default_rng(seed)
n = ds.n
orders = _feas_orders(widths, heights, rng)
# (0) Warm start: 上一可行解直接套算 (极快, 且已贴近可行边界)
if warm_seed is not None:
    res = pack_feasible(widths, heights, warm_seed["order"],
                        warm_seed["rot"], L, ideal_x=ideal_x)
    if res is not None and res[4] == 0:
        xs, ys, rw, rh, _ = res
        hpwl = net.total_hpwl(xs, ys, rw, rh)
        return "feasible", dict(xs=xs, ys=ys, rw=rw, rh=rh, hpwl=hpwl,
                                order=warm_seed["order"],
                                rot=warm_seed["rot"], overflow=0)
# L1 多源初解: 各排序 × {0%, 25% 旋转}, 取最低越界 (同时记录 order/rot)
best = None # (xs,ys,rw,rh,ovf,order,rot)
for key, order in orders.items():
    for rot_frac in (0.0, 0.25):
        rot = (rng.random(n) < rot_frac).astype(np.int64)
        res = pack_feasible(widths, heights, order, rot, L, ideal_x=ideal_x)
        if res is None:
            continue
        xs, ys, rw, rh, ovf = res
        if ovf == 0:
            hpwl = net.total_hpwl(xs, ys, rw, rh)
            return "feasible", dict(xs=xs, ys=ys, rw=rw, rh=rh, hpwl=hpwl,
                                    order=order, rot=rot, overflow=0)
        if best is None or ovf < best[4]:
            best = (xs, ys, rw, rh, ovf, order, rot)
# L2 Skyline-ILS: 固定 (L,L), 扰动 order/rot 把越界压到 0。
# ILS 起点优先取 warm seed (上一可行解, 已贴近边界), 其次多源最优。
if budget_ils is None:
    budget_ils = cfg.t_feas
seed_order, seed_rot = None, None
if warm_seed is not None:
    seed_order, seed_rot = warm_seed["order"], warm_seed["rot"]
elif best is not None:
    seed_order, seed_rot = best[5], best[6]
if seed_order is None:
    return "unknown", None
r2 = ils_feasible_refine(widths, heights, seed_order, seed_rot, net, L,
                        budget_ils, seed + 3)
if r2 is not None and r2["overflow"] == 0:
    return "feasible", r2
return "unknown", None

```

```

def ils_feasible_refine(widths, heights, order, rot, net, L, budget_s, seed):
    """Skyline + ILS 压越界到 0 (可行性精调 oracle) 。

    扰动 = 重插  $k=n//8$  模块 + 翻转少量旋转; 目标 = 越界量最小化 (溢出 0 即可行),
    Metropolis 接受 + 几何降温。
    """
    if order is None:
        return None
    rng = np.random.default_rng(seed)
    n = len(widths)
    widths = np.asarray(widths, dtype=np.int64)
    heights = np.asarray(heights, dtype=np.int64)
    bound_w = L + 5
    res0 = pack_skyline(order, rot, widths, heights, bound_w,
                       box=(L, L), auto_rotate=True)
    if res0 is None:
        return None
    xs, ys, rw, rh, W, H = res0
    order = list(order)
    rot = np.asarray(rot, dtype=np.int64).copy()
    ovf = max(0, int(W) - L) + max(0, int(H) - L)
    best = dict(xs=xs, ys=ys, rw=rw, rh=rh, hpwl=net.total_hpwl(xs, ys, rw, rh),
               order=order, rot=rot.copy(), overflow=ovf)
    T = 50.0
    alpha = 0.94
    start = time.time()
    it = 0
    while time.time() - start < budget_s and T > 0.1:
        inner = max(30, n * 2)
        for _ in range(inner):
            o2 = order[:]
            k = max(1, n // 8)
            idx = sorted(rng.choice(n, min(k, n), replace=False), reverse=True)
            vals = [o2[i] for i in idx]
            for v in vals:
                o2.remove(v)
            rng.shuffle(vals)
            posns = sorted(rng.integers(0, len(o2) + 1, size=len(vals)))
            for v, p in zip(vals, posns):
                o2.insert(p, v)
            r2 = rot.copy()
            flip = rng.choice(n, max(1, n // 12), replace=False)
            r2[flip] ^= 1
            res2 = pack_skyline(o2, r2, widths, heights, bound_w,
                               box=(L, L), auto_rotate=True)
            if res2 is None:
                continue
            xs2, ys2, rw2, rh2, W2, H2 = res2

```

```

    ovf2 =max(0, int(W2) -L) +max(0, int(H2) -L)
    it +=1
    if ovf2 <=ovf or rng.random() <np.exp(-(ovf2 -ovf) /T):
        order, rot =o2, r2
        ovf =ovf2
        if ovf2 ==0:
            hpwl =net.total_hpwl(xs2, ys2, rw2, rh2)
            return dict(xs=xs2, ys=ys2, rw=rw2, rh=rh2, hpwl=hpwl,
                        order=list(order), rot=rot.copy(), overflow=0)
        if ovf2 <best["overflow"]:
            best =dict(xs=xs2, ys=ys2, rw=rw2, rh=rh2,
                      hpwl=net.total_hpwl(xs2, ys2, rw2, rh2),
                      order=list(order), rot=rot.copy(), overflow=ovf2)

    T *=alpha
    best["iterations"] =it
    return best

# ----- #
# 整数二分 + 双证书
# ----- #
def binary_search_min_L(ds, net, widths, heights, cfg, L_lb, L_ub):
    """在整数域二分最小可行边长（可行性单调）。返回 (L_star, 可行布局, 日志)。

    增量收缩 warm start: 上一可行 L 的解 (order/rot) 原样作为下一更小 L 的
    初始种子, 避免每层 L 从头构造, 加速可行性边界精调。
    """
    rng =np.random.default_rng(cfg.seed)
    lb, ub =L_lb, L_ub
    cert =None # 当前上界对应的可行布局
    warm_seed =None # 上一可行 L 的解 (用于 warm start)
    steps =[]
    # 先直接探测 ub (问题二已证可行, 但拿布局当证书)
    status, layout =feasible_oracle(ds, widths, heights, net, ub, cfg,
                                    cfg.seed, budget_ils=cfg.t_feas)
    if status =="feasible":
        cert =layout
        warm_seed =dict(order=layout["order"], rot=layout["rot"])
    while lb <ub:
        mid =(lb +ub) //2
        status, layout =feasible_oracle(ds, widths, heights, net, mid, cfg,
                                       cfg.seed +mid *31,
                                       budget_ils=cfg.t_feas,
                                       warm_seed=warm_seed)
        steps.append((mid, status))
    if status =="feasible":
        ub =mid
        cert =layout

```

```

        warm_seed =dict(order=layout["order"], rot=layout["rot"])
    elif status == "infeasible":
        lb =mid +1
    else:
        # 未知: 启发式失败且 CP-SAT 未在时限内闭合 → 尝试子集不可行证书
        cp = "unknown"
        if ub -lb >=3: # gap 较大时才调用 CP-SAT, 避免浪费
            cp, cnt, el =cp_sat_subset_infeasible(ds, mid, cfg.cp_subset,
                                                  cfg.cp_timeout)

        if cp == "infeasible":
            lb =mid +1 # 子集不可行 → 整体不可行 (严格证书)
        else:
            # 无严格证书: 仅把探测点右移一位继续, 不把 lb 视为已证不可行下界
            # (最终 L*=ub 作为最佳已知可行上界报告, 不再称其已证最小)
            lb =mid +1
        # 兜底: 确保 ub 有可行证书
        if cert is None:
            status, layout =feasible_oracle(ds, widths, heights, net, ub, cfg,
                                            cfg.seed +999, budget_ils=cfg.t_feas)

            if status == "feasible":
                cert =layout
    return ub, cert, steps

# ----- #
# 主流程
# ----- #
def solve_dataset_q3(ds, cfg, net):
    widths =np.asarray(ds.widths, dtype=np.int64)
    heights =np.asarray(ds.heights, dtype=np.int64)
    A =ds.total_area
    L_lb =lower_bound_L(ds)
    L_ub =upper_bound_L(ds, 0.15)
    t0 =time.time()

    # ---- L1: 紧凑装箱初解 (获取 ideal_x 用 PeF 合法化加速 oracle) ----
    _, centers =None, None
    from hpwl import convex_lb_with_centers
    _, centers =convex_lb_with_centers(ds, L_ub)
    ideal_x =np.asarray([c[0] for c in centers], dtype=np.int64)

    # ---- L2: 整数二分 + 可行性证书 ----
    L_star, cert, steps =binary_search_min_L(
        ds, net, widths, heights, cfg, L_lb, L_ub)

    # ---- L3: 证书夹逼与 gap ----
    cp, cp_cnt, cp_el =cp_sat_subset_infeasible(ds, L_star, cfg.cp_subset,
                                                cfg.cp_timeout)

```

```

gap =gap_report_q3(ds, L_star, L_lb)
d_star =d_of_L(ds, L_star)

# ---- 第二阶段:  $L^*$  下更新 HPWL (从二分可行证书出发, Skyline-ILS 优化) ----
# 极紧轮廓 ( $d \leq 7-12\%$ ) 下  $L1$  初解池可能无一可行, 故直接用二分得到的
# 可行布局证书作为起点, 再跑 Skyline-ILS 压低 HPWL。
r2 =_hplw_at_L(ds, net, L_star, cfg, cert)
if r2 is None:
    # 回退: 仍尝试 Q2 完整管线
    q2_cfg =SCENARIOS["Q2"]
    q2_cfg.nproc =cfg.nproc
    q2_cfg.seed =cfg.seed
    q2_cfg.t_ils =cfg.t_ils
    q2_cfg.t_restart =cfg.t_restart
    q2_cfg.t_bstar =cfg.t_bstar
    q2_cfg.t_sp =cfg.t_sp
    q2_cfg.t_refine =cfg.t_refine
    q2_cfg.kp =cfg.kp
    r2 =solve_q2_dataset(ds, q2_cfg, net, Lhat=int(L_star))

# 对比基线: 问题二 ( $d=0.15$ ) 下 HPWL —— 优先读取已有结果, 否则同预算重算
q2_d15 =_load_q2_d15(ds.name, cfg.out_dir)
if q2_d15 is None:
    L15 =upper_bound_L(ds, 0.15)
    r15 =solve_q2_dataset(ds, q2_cfg, net, Lhat=int(L15))
    q2_d15 =r15["best"]["hplw"]
runtime =time.time() -t0

return dict(
    dataset=ds.name, A=A, L_lb=int(L_lb), L_ub0=int(L_ub),
    L_star=int(L_star), d_star=d_star, gap=gap,
    cp=(cp, cp_cnt, cp_el), steps=steps, cert=cert,
    hplw=r2["best"]["hplw"], hplw_lb=r2["hplw_lb"],
    hplw_gap=r2["gap"], feasible=r2["feasible"],
    q2_hplw=r2, q2_hplw_d15=q2_d15, runtime=runtime,
)

def _hplw_at_L(ds, net, L, cfg, cert):
    """在  $L \times L$  内从可行证书出发优化 HPWL (Skyline+ILS/SA, 多起点)。

    返回与 solve_q2_dataset 同构的 dict (best/sp/hplw_lb/gap/traces/...)。
    """
    from skyline_ils_q2 import ils_hplw_trajectory, pack_and_eval
    from sp_ga import sp_aga_solve
    from layer3_bound_q2 import lower_bound_hplw, check_feasible_q2, gap_report_q2
    widths =np.asarray(ds.widths, dtype=np.int64)
    heights =np.asarray(ds.heights, dtype=np.int64)

```

```

n = ds.n
tasks = []
# 起点1: 二分可行证书
if cert is not None:
    tasks.append((cert["order"], cert["rot"]))
# 起点2-4: L1 初解池中可行者 (若有)
rng = np.random.default_rng(cfg.seed)
pool = generate_initial_pool_q2(ds, net, L, rng, prob_rot=0.3)
for s in pool:
    if s["overflow"] == 0 and len(tasks) < cfg.kp:
        tasks.append((s["order"], s["rot"]))
# 起点5: 连接度排序
conn = {name: 0 for name in ds.names}
for net_list in ds.nets:
    for pin in net_list:
        if pin in conn:
            conn[pin] += 1
conn_order = sorted(range(n), key=lambda i: (-conn[ds.names[i]],
                                             -(ds.widths[i] * ds.heights[i])))
tasks.append((conn_order, np.zeros(n, dtype=np.int64)))

results = []
for i, (o, r) in enumerate(tasks):
    res = ils_hpwl_trajectory(widths, heights, o, r, net, L,
                              cfg.t_ils, seed=cfg.seed + i * 101 + 7)
    if res is not None:
        results.append(res)
if not results:
    return None
best = min(results, key=lambda r: r["hpwl"])
# 对照: SP+GA
sp_res = None
try:
    sp_res = sp_aga_solve(widths, heights, net, L, cfg.t_sp,
                          seed=cfg.seed + 7000)
except Exception:
    sp_res = None
hpwl_lb = lower_bound_hpwl(ds, L)
ok, max_ov, ovf = check_feasible_q2(ds, net, best["xs"], best["ys"],
                                     best["rw"], best["rh"], L)
gap = gap_report_q2(best["hpwl"], hpwl_lb)
return dict(
    best=best, sp=sp_res, hpwl_lb=hpwl_lb, feasible=ok,
    max_overlap=max_ov, overflow=ovf, gap=gap,
    traces=[r["trace"] for r in results],
    pool_hpwl=min((s["hpwl"] for s in pool), default=None),
    results_hpwl=[(r["hpwl"], 0) for r in results],
)

```



```

def _load_q2_d15(ds_name, out_dir="result"):
    """读取已生成的问题2 (d=0.15) HPWL 汇总 (若存在)。"""
    p = os.path.join(out_dir, "question 2", "q2_summary.json")
    try:
        with open(p, encoding="utf-8") as f:
            for row in json.load(f):
                if row.get("dataset") == ds_name:
                    return float(row.get("hpl"))
    except Exception:
        pass
    return None

def _worker_ils(task):
    from skyline_ils_q2 import ils_hpl_trajectory
    return ils_hpl_trajectory(**task)

def main():
    here = os.path.dirname(os.path.abspath(__file__))
    parser = argparse.ArgumentParser(description="问题三: 最小死区比例")
    parser.add_argument("--data-dir", default=os.path.normpath(os.path.join(here, "..", "附件")))
    parser.add_argument("--out-dir", default=os.path.normpath(os.path.join(here, "..", "result")))
    parser.add_argument("--datasets", default="n100,n200,n300")
    parser.add_argument("--nproc", type=int, default=6)
    parser.add_argument("--scale", type=float, default=1.0,
                        help="时间预算缩放系数 (快速测试用)")
    args = parser.parse_args()

    cfg = SCENARIOS["Q3"]
    cfg.nproc = args.nproc
    cfg.out_dir = args.out_dir
    cfg.t_feas *= args.scale
    cfg.t_ils *= args.scale
    cfg.t_restart *= args.scale
    cfg.t_bstar *= args.scale
    cfg.t_sp *= args.scale
    cfg.t_refine *= args.scale
    cfg.kp = max(2, int(round(cfg.kp * args.scale)))
    out_root = os.path.join(args.out_dir, "question 3")
    os.makedirs(out_root, exist_ok=True)

    datasets = [s.strip() for s in args.datasets.split(",") if s.strip()]
    results = []

```

```

for name in datasets:
    ds =load_dataset(args.data_dir, name)
    net =NetIndex(ds.names, ds.nets, ds.terminal_pos)
    print(f"\n==== 数据集 {name} {dataset_stats(ds)} =====", flush=True)
    r =solve_dataset_q3(ds, cfg, net)
    results.append(r)
    print(f" L_lb={r['L_lb']} L*={r['L_star']} "
          f"d*={r['d_star']*100:.3f}% 密度={r['gap']['density']*100:.2f}% "
          f"L*下总HPWL={r['hpwl']:,.0f} 可行={r['feasible']} "
          f"耗时={r['runtime']:,.1f}s", flush=True)

# ---- 结果表 ----
print("\n" + "=" * 116)
print("表1 问题三求解结果（最小死区比例）")
print("=" * 116)
print(f"{'数据集':<8}{'L_lb':>6}{'L*':>6}{'d*':>12}{'密度':>10}"
      f"{'L*下HPWL':>14}{'问题2 HPWL':>14}{'HPWL增幅':>10}{'可行':>6}")
for r in results:
    q2hp =r["q2_hpwl_d15"]
    inc =(r["hpwl"] -q2hp) /q2hp *100 if q2hp else 0.0
    print(f"{r['dataset']:<8}{r['L_lb']:>6}{r['L_star']:>6}"
          f"{r['d_star']*100:>11.3f}%{r['gap']['density']*100:>9.2f}%"
          f"{r['hpwl']:>14,.0f}{q2hp:>14,.0f}{inc:>9.2f}%"
          f"{'是' if r['feasible'] else '否':>6}")

# ---- 二分过程表 ----
print("\n" + "=" * 80)
print("表2 整数二分可行性探测过程")
print("=" * 80)
for r in results:
    s =", ".join(f"{m}:{'可行' if st=='feasible' else ('不可行' if st=='infeasible' else '未知')}"
                  for m, st in r["steps"])
    print(f"{r['dataset']:<8} {s}")

# ---- 可视化 ----
for r in results:
    ds =load_dataset(args.data_dir, r["dataset"])
    net =NetIndex(ds.names, ds.nets, ds.terminal_pos)
    b =r["q2_hpwl"]["best"]
    plot_layout_q2(
        ds, net, b["xs"], b["ys"], b["rw"], b["rh"], r["L_star"],
        os.path.join(out_root, f"q3_{r['dataset']}_layout.png"),
        title=f"问题三 {r['dataset']} 最小死区比例 d*={r['d_star']*100:.3f}%",
        hpwl=b["hpwl"], gap_pct=r["hpwl_gap"]["gap_pct"],
        feasible=r["feasible"])
    plot_convergence_q2(r["q2_hpwl"]["traces"], r["hpwl_lb"],
                        os.path.join(out_root, f"q3_{r['dataset']}_convergence.png"),

```

```

pool_hpwl=r["q2_hpwl"]["pool_hpwl"])

# ---- 保存结果 ----
summary =[]
for r in results:
    b =r["q2_hpwl"]["best"]
    ds =load_dataset(args.data_dir, r["dataset"])
    lines =[f"# {r['dataset']} Q3 placement L*={r['L_star']} "
            f"d*={r['d_star']:.4f} HPWL={b['hpwl']:.0f}"]
    for i in range(ds.n):
        lines.append(f"{ds.names[i]} {b['xs'][i]} {b['ys'][i]} "
                    f"{b['rw'][i]} {b['rh'][i]} {int(b['rot'][i])}")
    with open(os.path.join(out_root, f"q3_{r['dataset']}_placement.txt"),
              "w", encoding="utf-8") as fh:
        fh.write("\n".join(lines))
    summary.append(dict(
        dataset=r["dataset"], A=r["A"], L_lb=r["L_lb"], L_ub0=r["L_ub0"],
        L_star=r["L_star"], d_star=round(r["d_star"], 6),
        density=round(r["gap"]["density"], 6),
        gap_area=round(r["gap"]["gap_area"], 6),
        cp=(r["cp"][0], r["cp"][1]),
        hpwl=round(r["hpwl"], 1), hpwl_lb=round(r["hpwl_lb"], 1),
        hpwl_gap_pct=round(r["hpwl_gap"]["gap_pct"], 4),
        q2_hpwl_d15=round(r["q2_hpwl_d15"], 1) if r["q2_hpwl_d15"] else None,
        feasible=r["feasible"], runtime=round(r["runtime"], 1),
    ))
    with open(os.path.join(out_root, "q3_summary.json"), "w",
              encoding="utf-8") as fh:
        json.dump(summary, fh, ensure_ascii=False, indent=2)
    print(f"\n结果与可视化已输出至: {os.path.abspath(out_root)}")

if __name__ == "__main__":
    main()

```

附录

附录 1：问题四求解核心代码（复合矩形模块、C4 旋转与精确 DFS）

```
# 本程序及代码是在 AI 工具辅助下完成的。
# AI 工具名称: DeepSeek, 版本/型号: DeepSeek-V4-Flash, 开发机构/公司: 深度求索 (DeepSeek),
# 版本发布日期: 2026-08-08。
"""问题四核心: 复合矩形模块（矩形切割 + 刚性链接）表示、C4 旋转与复合 Skyline 装箱。

建模思想（《问题4_数学建模.md》修正1-6）：
* 每个异形模块切割为若干轴平行矩形子块，子块间通过刚性链接（硬约束 G1）
  保持固定相对位移 —— 子块绝对坐标 = 锚点 + 固定偏移，链接被烘进变量；
* C4 旋转群  $r \in \{0, 1, 2, 3\}$ ，旋转作用于整个模块，子块尺寸/偏移随旋转查表变换；
* 装箱时整模落位：放置一个模块即校验其全部子块占用格空闲且不越界。

数据结构：布尔占用位图 OCC,  $S \leq 2$  子块/模块、4 模块实例规模下  $O(nWH)$  向量化。
"""
from __future__ import annotations

import numpy as np

from packing import lex_key

class CompoundModule:
    """复合矩形模块：子块 (w, h, a, b)，其中 (a,b) 为相对模块锚点（左下角）的局部偏移。"""

    def __init__(self, name: str, subblocks):
        # subblocks: list of (w, h, a, b)
        self.name = name
        self.subblocks = [(int(w), int(h), int(a), int(b)) for w, h, a, b in
                           subblocks]

        self.area = sum(w * h for w, h, _, _ in self.subblocks)
        self.bbox = self._bbox(self.subblocks)

    @staticmethod
    def _bbox(subs):
        xs = [a for _, _, a, _ in subs] + [a + w for w, _, a, _ in subs]
        ys = [b for _, _, _, b in subs] + [b + h for _, h, _, b in subs]
        return max(xs) - min(xs), max(ys) - min(ys)

    # ----- #
    # C4 旋转（作用于整个模块）：子块尺寸与偏移随旋转查表变换
    # ----- #

    def rotated(self, r: int):
        """返回旋转  $r \in \{0, 1, 2, 3\}$  后的子块列表 [(w,h,a,b), ...]。"""
```

旋转后整体平移到非负象限，锚点仍为模块包围盒左下角。

90°CW: $(x,y) \rightarrow (y,-x)$; 180°: $(-x,-y)$; 270°CW: $(-y,x)$ 。

"""

if r ==0:

return list(self.subblocks)

对所有子块角点做 C4 变换

pts =[]

for w, h, a, b **in** self.subblocks:

for cx, cy **in** ((a, b), (a +w, b), (a +w, b +h), (a, b +h)):

if r ==1:

 pts.append((cy, -cx))

elif r ==2:

 pts.append((-cx, -cy))

else:

 pts.append((-cy, cx))

x0 =min(p[0] **for** p **in** pts)

y0 =min(p[1] **for** p **in** pts)

out =[]

for w, h, a, b **in** self.subblocks:

 corners =[]

for cx, cy **in** ((a, b), (a +w, b), (a +w, b +h), (a, b +h)):

if r ==1:

 corners.append((cy -x0, -cx -y0))

elif r ==2:

 corners.append((-cx -x0, -cy -y0))

else:

 corners.append((-cy -x0, cx -y0))

 xs =[p[0] **for** p **in** corners]

 ys =[p[1] **for** p **in** corners]

 out.append((max(xs) -min(xs), max(ys) -min(ys), min(xs), min(ys)))

return out

占用格 / 可行性判定

def fits(self, r: **int**, x: **int**, y: **int**, occ, W: **int**, H: **int**) ->**bool**:

 """锚点 (x,y) + 旋转 r 时，全部子块是否空闲且不越界。"""

for w, h, a, b **in** self.rotated(r):

 x0, y0 =x +a, y +b

if x0 <0 **or** y0 <0 **or** x0 +w >W **or** y0 +h >H:

return False

for px **in** range(x0, x0 +w):

for py **in** range(y0, y0 +h):

if (px, py) **in** occ:

return False

return True

def place(self, r: **int**, x: **int**, y: **int**, occ):

```

    for w, h, a, b in self.rotated(r):
        for px in range(x + a, x + a + w):
            for py in range(y + b, y + b + h):
                occ.add((px, py))

def unplace(self, r: int, x: int, y: int, occ):
    for w, h, a, b in self.rotated(r):
        for px in range(x + a, x + a + w):
            for py in range(y + b, y + b + h):
                occ.discard((px, py))

def occupied_cells(self, r: int, x: int, y: int) -> set:
    s = set()
    for w, h, a, b in self.rotated(r):
        for px in range(x + a, x + a + w):
            for py in range(y + b, y + b + h):
                s.add((px, py))
    return s

# ----- #
# 复合 Skyline 装箱 (L1/L2 共用)
# ----- #
def pack_compound(modules, order, rot, bound_w, bound_h, auto_rotate=False):
    """复合 Skyline 装箱: 整模落位, 返回 (xs, ys, rot_used, W, H) 或 None。

    Args:
        modules: list[CompoundModule]
        order: 模块序号放置顺序
        rot: per-module 旋转 (rot{0,1,2,3}), auto_rotate=True 时被覆盖
        bound_w/bound_h: 扫描范围上界
        auto_rotate: 每个模块在 C4 中贪心选"最低、最左"落位。
    """
    n = len(order)
    occ = set()
    xs = [0] * n
    ys = [0] * n
    rot_used = [0] * n
    for m in order:
        mod = modules[m]
        r_range = range(4) if auto_rotate else [int(rot[m])]
        best = None
        for r in r_range:
            rsubs = mod.rotated(r)
            max_x = max(a + w for w, h, a, b in rsubs)
            max_y = max(b + h for w, h, a, b in rsubs)
            # 最低、最左: 逐 y 行找最左可行 x
            for y in range(max(0, bound_h - max_y) + 1):

```

```

        found =None
        for x in range(max(0, bound_w -max_x) +1):
            if mod.fits(r, x, y, occ, bound_w, bound_h):
                found =x
                break
        if found is not None:
            cand =(y, found, r)
            if best is None or cand[:2] <best[:2]:
                best =cand
            break
    if best is None:
        return None
    y, x, r =best
    xs[m], ys[m], rot_used[m] =x, y, r
    mod.place(r, x, y, occ)
W =int(max(xs[m] +a +w for m in range(n)
            for w, h, a, b in modules[m].rotated(rot_used[m])))
H =int(max(ys[m] +b +h for m in range(n)
            for w, h, a, b in modules[m].rotated(rot_used[m])))
    return xs, ys, rot_used, W, H

def bbox_of_modules(modules):
    """每个模块的包围盒矩形 (w, h) (B*-树对照用)。"""
    return [(m.bbox[0], m.bbox[1]) for m in modules]

def lex_cost_q4(W: int, H: int) ->int:
    """字典序目标: 面积最小 → 长宽比接近 1 (与问题1一致)。"""
    key =lex_key(W, H)
    return key[0] *10_000_000 +key[1]

```

本程序及代码是在 AI 工具辅助下完成的。

AI 工具名称: DeepSeek, 版本/型号: DeepSeek-V4-Flash, 开发机构/公司: 深度求索(DeepSeek),
版本发布日期: 2026-08-08。

"""问题四 L3: 精确验证 —— 面积下界 + 因子枚举 + 首空位精确 DFS (4 模块实例)。

思路 (《问题4_算法.md》第五节):

- * 面积下界 $S^* \geq A$; 在 A 的整数因子对 (W, H) 上检验可铺砌性;
- * 首空位规范序 DFS: 始终把下一模块放在"最左下首空闲格", 锚点置于覆盖该格子的子块对应位置 (整体落位, 刚性链接天然满足);
- * 剪枝: 剩余模块面积 $>$ 当前空格数则回溯; 旋转/镜像等价剪枝。

"""

```
from __future__ import annotations
```

```
import math
```

```

def factor_pairs(A: int):
    """返回 A 的所有整数因子对 (W,H)，按"面积升序、长宽比接近1"排序。"""
    pairs = []
    for w in range(1, int(math.isqrt(A)) + 1):
        if A % w == 0:
            h = A // w
            pairs.append((w, h))
            pairs.append((h, w))
    pairs = list(dict.fromkeys(pairs))
    pairs.sort(key=lambda p: (p[0] * p[1], abs(p[0] / p[1] - 1.0)))
    return pairs

def _first_free_cell(occ, W, H):
    for y in range(H):
        for x in range(W):
            if (x, y) not in occ:
                return x, y
    return None

def exact_pack(modules, W, H):
    """首空位规范序 DFS: 判定 modules 能否铺满 W×H。

    Returns: (ok, solution) solution = {m: (x, y, r)} 或 None。
    """
    n = len(modules)
    total_area = sum(m.area for m in modules)
    if total_area > W * H:
        return False, None

    occ = set()
    placed = [False] * n
    sol = {}
    result = [False, None]

    def dfs(n_placed):
        if result[0]:
            return True
        if n_placed == n:
            result[0] = True
            result[1] = dict(sol)
            return True
        cell = _first_free_cell(occ, W, H)
        if cell is None:
            return False
        cx, cy = cell
        # 剩余面积剪枝

```



```

remain =sum(modules[m].area for m in range(n) if not placed[m])
if remain >W *H -len(occ):
    return False
for m in range(n):
    if placed[m]:
        continue
    mod =modules[m]
    for r in range(4):
        for w, h, a, b in mod.rotated(r):
            # 锚点 = cell 相对子块 (a,b) 的反推
            ax, ay =cx -a, cy -b
            if not (0 <=ax and 0 <=ay and ax +w <=W and ay +h <=H):
                continue
            if not mod.fits(r, ax, ay, occ, W, H):
                continue
            placed[m] =True
            sol[m] =(ax, ay, r)
            mod.place(r, ax, ay, occ)
            if dfs(n_placed +1):
                return True
            # 回退
            mod.unplace(r, ax, ay, occ)
            placed[m] =False
            del sol[m]
    return False

dfs(0)
return result[0], result[1]

def find_optimal(modules):
    """因子枚举 + 精确 DFS: 返回 (S*, W, H, solution, trials)。

    trials: [(W,H,可行?)] 尝试历史。
    """
    A =sum(m.area for m in modules)
    trials =[]
    for W, H in factor_pairs(A):
        ok, sol =exact_pack(modules, W, H)
        trials.append((W, H, ok))
        if ok:
            return W *H, W, H, sol, trials
    # 无完美铺砌: 放宽到最小面积 > A (本实例不会走到)
    for W in range(1, A +1):
        for H in range(W, A +1):
            if W *H <A:
                continue
            ok, sol =exact_pack(modules, W, H)

```

```
    trials.append((W, H, ok))  
    if ok:  
        return W * H, W, H, sol, trials  
    return None, None, None, None, trials
```